Reconciling Abstraction with High Performance: A MetaOCaml approach

Oleg Kiselyov
Tohoku University, Japan
oleg@okmij.org

now
the essence of knowledge
Boston — Delft

Foundations and Trends® in Programming Languages

Published, sold and distributed by:
now Publishers Inc.
PO Box 1024
Hanover, MA 02339
United States
Tel. +1-781-985-4510
www.nowpublishers.com
sales@nowpublishers.com

Outside North America:
now Publishers Inc.
PO Box 179
2600 AD Delft
The Netherlands
Tel. +31-6-51115274

The preferred citation for this publication is

O. Kiselyov. *Reconciling Abstraction with High Performance: A MetaOCaml approach.* Foundations and Trends® in Programming Languages, vol. 5, no. 1, pp. 1–101, 2018.

This Foundations and Trends® issue was typeset in LaTeX using a class file designed by Neal Parikh. Printed on acid-free paper.

ISBN: 978-1-68083-436-9
© 2018 O. Kiselyov

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, mechanical, photocopying, recording or otherwise, without prior written permission of the publishers.

Photocopying. In the USA: This journal is registered at the Copyright Clearance Center, Inc., 222 Rosewood Drive, Danvers, MA 01923. Authorization to photocopy items for internal or personal use, or the internal or personal use of specific clients, is granted by now Publishers Inc for users registered with the Copyright Clearance Center (CCC). The 'services' for users can be found on the internet at: www.copyright.com

For those organizations that have been granted a photocopy license, a separate system of payment has been arranged. Authorization does not extend to other kinds of copying, such as that for general distribution, for advertising or promotional purposes, for creating new collective works, or for resale. In the rest of the world: Permission to photocopy must be obtained from the copyright owner. Please apply to now Publishers Inc., PO Box 1024, Hanover, MA 02339, USA; Tel. +1 781 871 0245; www.nowpublishers.com; sales@nowpublishers.com

now Publishers Inc. has an exclusive license to publish this material worldwide. Permission to use this content must be obtained from the copyright license holder. Please apply to now Publishers, PO Box 179, 2600 AD Delft, The Netherlands, www.nowpublishers.com; e-mail: sales@nowpublishers.com

Foundations and Trends® in Programming Languages

Volume 5, Issue 1, 2018

Editorial Board

Editor-in-Chief

Mooly Sagiv
Tel Aviv University
Israel

Editors

Martín Abadi
Google & UC Santa Cruz

Anindya Banerjee
IMDEA

Patrick Cousot
ENS Paris & NYU

Oege De Moor
University of Oxford

Matthias Felleisen
Northeastern University

John Field
Google

Cormac Flanagan
UC Santa Cruz

Philippa Gardner
Imperial College

Andrew Gordon
Microsoft Research & University of Edinburgh

Dan Grossman
University of Washington

Robert Harper
CMU

Tim Harris
Oracle

Fritz Henglein
University of Copenhagen

Rupak Majumdar
MPI-SWS & UCLA

Kenneth McMillan
Microsoft Research

J. Eliot B. Moss
UMass, Amherst

Andrew C. Myers
Cornell University

Hanne Riis Nielson
TU Denmark

Peter O'Hearn
UCL

Benjamin C. Pierce
UPenn

Andrew Pitts
University of Cambridge

Ganesan Ramalingam
Microsoft Research

Mooly Sagiv
Tel Aviv University

Davide Sangiorgi
University of Bologna

David Schmidt
Kansas State University

Peter Sewell
University of Cambridge

Scott Stoller
Stony Brook University

Peter Stuckey
University of Melbourne

Jan Vitek
Purdue University

Philip Wadler
University of Edinburgh

David Walker
Princeton University

Stephanie Weirich
UPenn

Editorial Scope

Topics

Foundations and Trends® in Programming Languages publishes survey and tutorial articles in the following topics:

- Abstract interpretation
- Compilation and interpretation techniques
- Domain specific languages
- Formal semantics, including lambda calculi, process calculi, and process algebra
- Language paradigms
- Mechanical proof checking
- Memory management
- Partial evaluation
- Program logic
- Programming language implementation
- Programming language security
- Programming languages for concurrency
- Programming languages for parallelism
- Program synthesis
- Program transformations and optimizations
- Program verification
- Runtime techniques for programming languages
- Software model checking
- Static and dynamic program analysis
- Type theory and type systems

Information for Librarians

Foundations and Trends® in Programming Languages, 2018, Volume 5, 4 issues. ISSN paper version 2325-1107. ISSN online version 2325-1131. Also available as a combined paper and online subscription.

Foundations and Trends® in Programming Languages
Vol. 5, No. 1 (2018) 1–101
© 2018 O. Kiselyov
DOI: 10.1561/2500000038

Reconciling Abstraction with High Performance: A MetaOCaml approach

Oleg Kiselyov
Tohoku University, Japan
oleg@okmij.org

Contents

1 Introduction **2**

1.1 Why metaprogramming? 2
1.2 Why this tutorial? 4
1.3 Why MetaOCaml? 5
1.4 Overview 7
1.5 Obtaining MetaOCaml 8

2 First Steps **9**

2.1 Now or later 9
2.2 Power 12
2.3 Offline code generation 17
2.4 Runtime specialization and its benchmark 18
2.5 Recap 20
2.6 A historical aside 21

3 Filtering **23**

3.1 Specializing to the known filter order 26
3.2 Specialization to the known coefficients 30
3.3 Smarter specialization 35
3.4 Further challenges 38
3.5 Recap 39

4 Linear Algebra DSL: Complex Vector Arithmetic and Data Layout **40**
4.1 Data layout problem 41
4.2 Abstracting arithmetic 43
4.3 Abstracting vectors 48
4.4 Vector arithmetic DSL 50
4.5 Compiling vector DSL 51
4.6 Recap and further challenges 56

5 Linear Algebra DSL: Matrix-Vector Operations and Modular Optimizations **58**
5.1 Shonan challenge 1 59
5.2 BLAS 2 DSL 60
5.3 Implementing and generating matrix-vector multiplication . 63
5.4 Specializing to the known dimensions 66
5.5 Specializing to the known matrix: Partially-known values . 68
5.6 Algebraic simplifications 73
5.7 Selective unrolling 75
5.8 Cross-stage persistence for large data 76
5.9 Recap 81

6 From an Interpreter to a Compiler: DSL for Image Manipulation **82**
6.1 Image-processing DSL 82
6.2 Interpreting DSL 83
6.3 Compiling DSL 86

7 Further Challenges **89**
7.1 Digital filters 89
7.2 Linear Algebra DSL 89
7.3 Other Challenges 90

8 Conclusions **92**

Acknowledgements **94**

Index of the Accompanying Code **95**

References **97**

Abstract

A common application of generative programming is building high-performance computational kernels highly tuned to the problem at hand. A typical linear algebra kernel is specialized to the numerical domain (rational, float, double, etc.), loop unrolling factors, array layout and a priori knowledge (e.g., the matrix being positive definite). It is tedious and error prone to specialize by hand, writing numerous variations of the same algorithm.

The widely used generators such as ATLAS and SPIRAL reliably produce highly tuned specialized code but are difficult to extend. In ATLAS, which generates code using printf, even balancing parentheses is a challenge. According to the ATLAS creator, debugging is nightmare.

A typed staged programming language such as MetaOCaml lets us state a general, obviously correct algorithm and add layers of specializations in a modular way. By ensuring that the generated code always compiles and letting us quickly test it, MetaOCaml makes writing generators less daunting and more productive.

The readers will see it for themselves in this hands-on tutorial. Assuming no prior knowledge of MetaOCaml and only a basic familiarity with functional programming, we will eventually implement a simple domain-specific language (DSL) for linear algebra, with layers of optimizations for sparsity and memory layout of matrices and vectors, and their algebraic properties. We will generate optimal BLAS kernels. We shall get the taste of the "Abstraction without guilt".

O. Kiselyov. *Reconciling Abstraction with High Performance: A MetaOCaml approach.* Foundations and Trends® in Programming Languages, vol. 5, no. 1, pp. 1–101, 2018.

DOI: 10.1561/2500000038.

1

Introduction

1.1 Why metaprogramming?

Ever-present in all areas of programming is the agonizing trade-off between, on one hand, the maintainable, reusable, easy to read and understand, obviously correct, textbook code – and the code that performs well. The trade-off is exacerbated in high-performance computing (HPC). Coding the matrix-vector multiplication just as `a * v` is clear, portable, self-describing. On the other hand, the typical high-performance code that multiplies an integer-valued matrix to an integer-valued vector takes many, many lines and not at all self-evident. It looks nothing like `a * v`. It also looks nothing like the code that multiplies a single-precision floating-point matrix to a floating-point vector. Which, in turn, bears scarcely any resemblance to the high-performance code multiplying a sparse matrix to a vector.

Already at the end of the last century it was recognized that we can no longer rely on optimizing compilers to turn the high-level code to the high-performance code (see references in Cohen et al. (2006)): many profitable optimizations are domain specific and often narrowly applicable, and hence unlikely to be supported by a general-purpose compiler. Even the simplest replacement `0*e` with `0` is not generally

sound: think of e that calls external functions or returns NaN[1]. A domain expert, knowing the input data and the entire algorithm, could tell that the side effects of e may be disregarded or NaNs do not occur – hence the optimization should be carried out, for particular multiplications in particular expressions.

It is cognitively and economically prohibitive for general-purpose compilers to give programmers such minute level of control over optimizations. It is very common therefore for experts to write the computational kernels by hand – and keep re-writing them to accommodate new architectures or new patterns in the input data.

Metaprogramming – code generation specifically – promises a way out: instead of a program we write a program generator, which incorporates domain-specific knowledge and outputs a number of low-level, specialized, high-performance programs. This is the approach taken by the widely known and used fast Fourier transformer generator FFTW (Frigo and Johnson, 2005), basic linear algebra (BLAS) generator ATLAS (Whaley and Petitet, 2005), DSP and linear algebra generator SPIRAL (Püschel et al., 2005), image filter generator Halide (Ragan-Kelley et al., 2013).

The above projects also showed that writing a good generator is still very difficult: it is worth a paper in a prestigious conference. For example, ATLAS – which uses C to generate C code as strings – has been notoriously difficult to write, debug and extend. We need help with code-generating chores – provided by MetaOCaml, Lightweight Modular Staging in Scala (Rompf and Odersky, 2012) or Template Haskell (Sheard and Peyton Jones, 2002). We need levels of abstractions.

Ideally, the end user would write the matrix-vector multiplication *generator* just as a ∗ v. The (domain-specific) operation ∗ would be implemented (perhaps by another programmer, an algorithm designer) using the vocabulary of a different, 'MapReduce' domain:

```
let dot v1 v2 =  reduce add zero (zip_with mul v1 v2)
let ( * ) a v =  map (dot v) a
```

The generators reduce, add, etc. are to be provided by some other do-

[1]In fact, OCaml before version 4.05 incorrectly performed this optimization: `https://github.com/ocaml/ocaml/pull/956`.

main expert, a specialist in data layout. An expert in the domain over which matrices and vectors are taken would supply a library of algebraic laws, to invoke to simplify scalar expressions. Eventually it comes to MetaOCaml, to generate code in OCaml or (with offshoring) C or LLVM. This ideal is attainable! In fact, by the end of the tutorial, we shall implement exactly such layered domain-specific language for simple linear algebra. Rompf et al. (2013) and the FEniCS project (Markall et al., 2013) present more examples of such generator DSLs built by composing progressively more detailed abstractions – and their empirical evaluation.

All in all, we do let the end users write programs in the clearest to them form in terms of the familiar domain vocabulary – and yet obtain the high-performance code tuned to various domains. To use Ken Kennedy's phrase, metaprogramming gives us "Abstraction without guilt".

1.2 Why this tutorial?

The goal of the tutorial is to teach how to write typed code generators, how to make them modular, and how to gradually introduce domain-specific optimizations – with MetaOCaml. By the end of the tutorial we will implement a simple domain-specific language (DSL) for linear algebra, with layers of optimizations for the memory layout of matrices and vectors, their sparsity and algebraic properties. We will generate optimal Basic Linear Algebra (BLAS) kernels. Hopefully the readers will see that writing generators is not too complicated and that (staged) types are of great help.

The readers are not expected to know MetaOCaml but should be somewhat familiar with a modern functional language. Even a brief experience with a language in the ML family is a boon. However, Scala or Haskell, etc., programmers should not feel left out.

The present tutorial is by and large a written record of a live tutorial delivered on several occasions (first at CUFP – Commercial Users of Functional Programming 2013). It inherits the hands-on style of those tutorials, built around live coding, in interaction with the MetaOCaml

and its type checker – and the audience. We will be developing code piece-by-piece, by submitting small fragments to the MetaOCaml interpreter; fixing the pointed out type problems; generating sample code and testing it; noting the points of improvement and adjusting the generator as needed. The tutorial includes many exercises and homework projects to work on alone or in groups.

1.3 Why MetaOCaml?

We will be using *BER MetaOCaml* (Kiselyov, 2017, 2014), which is a complete re-implementation of the no longer available original MetaOCaml by Walid Taha, Cristiano Calcagno and collaborators (Calcagno et al., 2003).

BER MetaOCaml is a conservative extension of OCaml for "writing programs that generate programs". BER MetaOCaml adds to OCaml the type of *code values* (denoting "program code", or future-stage computations), and two basic constructs to build them: quoting and splicing. The generated code can be printed, stored in a file – or compiled and linked-back to the running program, thus implementing run-time code optimization. MetaOCaml code without staging annotations, or with the annotations erased, is regular OCaml.

MetaOCaml has been successfully used for the most optimal stream fusion (Kiselyov et al., 2017), specializing numeric and dynamic programming algorithms, building FFT kernels, compilers for an image processing and database query DSLs, OCaml server pages, generating families of specialized basic linear algebra and Gaussian Elimination routines, and high-performance stencil computations (Aktemur et al., 2013). See Lengauer and Taha (2006) for a collection of MetaOCaml applications.

Writing code generators in a typed staged language like MetaOCaml benefits in several ways. First, the generated code will be well-formed, with all parentheses matching. Such a guarantee is a dear wish when writing C with printf (as done in ATLAS) or C++ with Matlab. MetaOCaml makes sure that the generated code is well-typed and shall compile without errors. There is no longer puzzling out a compilation

error in the generated code, which is typically large, obfuscated and with unhelpful variable names. Mainly, code generation errors are reported in terms of the generator rather than the generated code. The tutorial will give many chances to see the importance of good error reporting.

MetaOCaml generators are hygienic, producing well-scoped code, with no unbound variables. Otherwise, hygiene violations are hard to detect in practice and may lead to the devious error of unintentionally bound variables. Although the unbound variables in the generated code stand out (when compiling it), determining what has caused them proved to be highly non-trivial in practice, as reported by Ofenbeck et al. (2016). The authors wrote a new compiler testing framework, to specifically detect unbound variable and other such problems introduced during refactoring of generators. MetaOCaml is designed to prevent the generation of the problematic code in the first place.

Most importantly, MetaOCaml is typed. Types, staged types in particular, really do help write the code. All throughout the tutorial we will be writing code in live interaction with the type checker – accepting type errors not as a punishment but as a valuable hint. We shall see on many occasions that once we fix the type signature, the generator practically writes itself. The type checker will tell us where to put a staging annotation.

MetaOCaml is purely generative: the generated code is treated as a black box and cannot be examined. One can put code together but cannot take it apart. Pure generativity significantly simplifies the type system and strengthens the static assurances. It may also seem that pure generativity precludes code optimizations. Fortunately, that is not the case, as shall soon see.

The staging annotations of MetaOCaml are like the "assembler" instructions of metaprogramming. We need higher-level abstractions. The final benefit of MetaOCaml – compared to the preprocessors like camlp4 or ppx – is that it is part of OCaml itself, and hence can take the full advantage of OCaml's abstraction and combination facilities, from higher-order functions to modules. Building optimization libraries and composing generators is the stress of the tutorial.

1.4 Overview

The tutorial is based on the progression of problems, which, except the introductory one, are all slightly simplified versions of real-life problems:

1. First steps in staging and MetaOCaml
2. Digital filters
3. Complex vector multiplication: varying data representation (structure of arrays vs. array of structures)
4. Systematic optimization of simple linear algebra: building extensively specialized general BLAS
5. From an interpreter to a compiler: DSL for image manipulation
6. Further challenges (Homework)

In fact, problems 3, 4 and 6 were suggested by HPC researchers as challenges to program generation community (Shonan challenges). The common theme is building high-performance computational kernels highly tuned to the problem at hand. Hence most problems revolve around simple linear algebra – a typical and most frequently executed part in HPC.

The stress on high-performance applications and on modular optimizations and generators sets this tutorial apart from Taha's very accessible, gentle introductions to the 'classical' partial evaluation and staging, focused on turning an interpreter of a generally higher-order language into a compiler (Taha, 2004, 2008). We also get to see this classical area in §6; however, we pay less attention to lambda-calculus and more to image processing. Furthermore, this tutorial mentions recent additions to MetaOCaml such as offshoring and let-insertion.

The source code for the tutorial is available as a supplement: §8.

1.5 Obtaining MetaOCaml

The tutorial needs at least BER MetaOCaml N104, which is available from OPAM

```
opam update
opam switch 4.04.0+BER  # or a later version
eval `opam config env`
```

The MetaOCaml web page `http://okmij.org/ftp/ML/MetaOCaml.html` talks in depth about the design, implementation and history of MetaOCaml. It also shows other ways of installing it.

2

First Steps

2.1 Now or later

As a quick spot-check that MetaOCaml is properly installed and functional, let's start metaocaml (the interactive interpreter: the top-level) and enter at its prompt

```
let v = 1+2;;
⇝ val v : int = 3
```

The double-semicolons (which we will often elide thereafter) signify the end of input in the (Meta)OCaml top-level. The response of the interpreter is shown after ⇝. With no staging annotations in sight, MetaOCaml behaves identically to the ordinary OCaml. In our example, the addition expression is evaluated, now, and the result is bound to the variable v.

With the staging annotation .< ... >., called *brackets*,

```
let c = .<1 + 2>.;;
⇝ val c : int code = .<1 + 2>.
```

we no longer see any number 3 because no addition is performed *now*. In fact, nothing is performed now: .<1+2>. is bound to the identifier

c as it is. Brackets quote[1] the containing expression, annotating it as computed *later*, when the generated code is later run. For now, .<1+2>. is a value, and, as such, can be bound to identifiers, passed as arguments to and returned as results from functions. It is called a *code value*. In the first approximation, one may think of a code value as an abstract syntax tree for the generated program; one may even take, just for now, .<1+2>. as the string "1+2". (Free variables, which appear in §2.2, demand a more complicated representation.) The response from the interpreter shows that code values have a characteristic type: for an expression e of type t, .<e>. is a value of the type t code. The (closed) code values can be printed, revealing the quoted expression.

The remaining annotation is the so-called *escape* .~e, which must be used within brackets. It tells that e is computed now but produces the code to run later. That code is spliced-in into the containing bracket:

```
.<.~c * 10>.;;
⇝ - : int code =  .<(1 + 2) * 10>.
```

In our example, the expression .< .~(.<1+2>.) * 10 >. is evaluated to produce the code value .<(1 + 2) * 10>.: as if the escape 'dissolves' the brackets freeing the containing expression. In Lisp terminology (Bawden, 1999), brackets 'quasi-quote' and escape 'unquotes'[2]. The expression unquoted by the escape still appears as a sub-expression, not just a sub-string, in a larger quotation: therefore, the printed code value has parentheses around the spliced 1+2.

Exercise 1. What would happen if there were no parentheses?

In the previous examples the escape is applied to a variable or a value. In general, we can 'escape' to an arbitrary expression that could produce a code value:

```
.< .~( .<.~c * 10>. ) * 10 >.;;
```

The outer escape is applied to .<.~c * 10>., which is not a value since it itself has an escape.

[1] Actually, brackets 'quasi-quote', using Lisp terminology – as we are about to see

[2] We shall see soon that MetaOCaml sharply differs from metaprogramming in Lisp when it comes to code with bound variables. Unlike Lisp, MetaOCaml is hygienic.

Exercise 2. We did not show the result of the last example. Guess it first, and then check with MetaOCaml.

Let us generalize and abstract over the escape expression:

```
let times10 x = .< .~x * 10 >.
⇝ val times10 : int code → int code = <fun>
```

Thus defined function is essentially a code template, with the 'hole' x. Applying the function to a code value times10 c fills the hole, generating .<(1 + 2) ∗ 10>. seen earlier. We can also do

```
.< .~(times10 c) * 10 >.
```

and even

```
times10 (times10 c)
```

The last code fragment is a generator with no overt staging annotations: they are hidden behind the earlier defined code fragment c and the code template times10. With properly chosen abstractions, generators look and act like the ordinary code we commonly write. We should not be afraid of metaprogramming.

Although we have not paid much attention to types and never entered them, they do exists. If we write

```
.<.~c *. 10.>.
⇝ .<.~c *. 10.>.
        ^
Error: This expression has type int code
       but an expression was expected of type float code
       Type int is not compatible with type float
```

we get a type error. Had the example been accepted and evaluated, it would have produced an ill-typed code value .< (1+2) ∗. 10. >.. Compiling the the quoted (generated) code would have raised a type error: an attempt to apply a floating-point multiplication ∗. to an integer expression. (In OCaml, arithmetic operations are not overloaded and no implicit conversions are performed.) But it does not come to that: a generator that could produce an ill-typed code does not even type-check (let alone run).

The type checker may reject not only a ready-to-run generator but also a not-yet-filled code template:

```
fun x → .<if .~x then .~x+1 else 10>.
⇝ fun x → .<if .~x then .~x+1 else 10>.
                         ^
Error: This expression has type bool code
       but an expression was expected of type int code
       Type bool is not compatible with type int
```

because otherwise it may generate an ill-typed code. In fact, it always does.

Exercise 3. Try to instantiate the template – manually replace .~x within brackets with some code value – and see what sort of code comes out

MetaOCaml does not wait and see what happens when the template is instantiated: it is rejected right away, and the error message points the problem with the use of the template argument, no matter what it may be. All in all, although the generated code may be compiled and run later, it is always type-checked now. Therefore, *a well-typed generator generates only well-typed code.*

The stage at which the evaluation occurs – now – is often called in the literature *the present stage*. A later stage is called a *future stage*. MetaOCaml, as a multi-stage language, supports an arbitrary number of future stages, letting us write not only code generators but also generators of code generators, etc. More than one future stage, however, hardly ever occurs in practice (and not at all in the present tutorial).

We have not yet discussed how to actually compile and run the generated code. We shall see it in a bigger example, of using staging for specialization.

2.2 Power

Albeit becoming a cliche, the power example is still one of the fastest ways to introduce metaprogramming[3]. The power function computes x^n:

```
let square x = x * x
```

[3]It was the running example in the founding paper on partial evaluation. See §2.6 for the history.

```
let rec power : int → int → int = fun n x →
  if n = 0 then 1
  else if n mod 2 = 0 then square (power (n/2) x)
  else x ∗ (power (n−1) x)
```

We have explicitly attached the type signature, although it is not needed: it will be inferred. Still, giving signatures to top-level functions is generally a good idea: it makes the code easier to write and later understand, and it makes error messages clearer. In metaprogramming, writing signatures helps to place staging annotations (sometimes, to the point of automation), as we shall see momentarily. As a quick spot check,

```
power 7 2;;
⇝ − : int = 128
```

Suppose our program has to compute x^7 many times. We may define

```
let power7 x = power 7 x
```

to name and share the partial application.

In MetaOCaml, we may also *specialize* the power function to a particular value n, obtaining the code which will later receive x and compute x^n. We shall re-write power n x and annotate expressions as computed 'now' (when n is known) or 'later' (when x is given). Let us do that gradually.

As the first attempt, we copy-and-paste the earlier power code, changing the name to spower and modifying the type signature to tell which arguments are known now (they have the ordinary types like int) and which arguments will be known only later. The latter have types like int code and cannot be immediately computed with. The result of spower will also have the type int code.

Exercise 4. Why is the result type int code?

We have not yet made any adjustments to the body (except for renaming power to spower):

```
let rec spower : int → int code → int code = fun n x →
  if n = 0 then 1
  else if n mod 2 = 0 then square (spower (n/2) x)
  else x ∗ (spower (n−1) x)
```

No wonder then that the type checker complains, since the signature and the body of the function no longer correspond:

```
⇝     Characters 74–75:
  if n =  0 then 1
                 ^
Error: This expression has type int but an expression was expected of type
       int code
```

We should take the complaint seriously: it says that the type checker, instead of 1 expected to see something of the type int code. To satisfy the expectation, we just have to enclose 1 into brackets, which gives the expression of the int code type. Hence, the second version of the code (We highlight the changes):

```
let rec spower : int → int code → int code =  fun n x →
  if n =  0 then .<1>.
  else if n mod 2 =  0 then square (spower (n/2) x)
  else x ∗ (spower (n−1) x)
;;
⇝    Characters 114–130:
  else if n mod 2 =  0 then square (spower (n/2) x)

Error: This expression has type int code
       but an expression was expected of type int
```

The type-checker complains again, at a different place though: The spower generates the code that will give an int later, but square needs an integer now.

Exercise 5. What do you think we should do?

Here is one way to resolve the mismatch: Since we cannot give square an int now, we have to delay the whole application, until later:

```
let rec spower : int → int code → int code =  fun n x →
  if n =  0 then .<1>.
  else if n mod 2 =  0 then .< square .~(spower (n/2) x) >.
  else x ∗ (spower (n−1) x)
;;
⇝   Characters 146–147:
  else x ∗ (spower (n−1) x)
         ^
Error: This expression has type int code
       but an expression was expected of type int
```

The type checker points out that we have to likewise delay the multiplication by x, whose int code type tells it is not available now. The next version of the code:

```
let rec spower : int → int code → int code = fun n x →
  if n = 0 then .<1>.
  else if n mod 2 = 0 then .< square .~(spower (n/2) x) >.
  else .<.~x * (spower (n−1) x)>.
;;
    Characters 154−170:
  else .<.~x * (spower (n−1) x)>.
```

```
Error: This expression has type int code
       but an expression was expected of type int
```

is still lacking: We have forgotten to escape the spower invocation. If an e has the type int code, then .~e clearly has the type int (the escape dissolves the brackets, remember?) and the type checker should be satisfied:

```
let rec spower : int → int code → int code = fun n x →
  if n = 0 then .<1>.
  else if n mod 2 = 0 then .< square .~(spower (n/2) x) >.
  else .<.~x * .~(spower (n−1) x)>.
;;
⇝ val spower : int → int code → int code = <fun>
```

It is indeed. We have successfully turned power to a code generator: just by adjusting the signature and adding annotations at the places the type-checker complained about. The types literally helped us write the code.

We are not completely done, however. Consider the specialization spower 7, which has the type int code → int code. This is the type of a code template. A code value that describes a future-stage function that will receive x and compute x^7 has the different type, (int→int) code. It is rather easy, however, to convert the value of the former type to the latter. Types again help: if we need something of the type t code, we have to find an expression of the type t and enclose it in brackets. Hence, in the first approximation:

```
let spowern : int → (int → int) code = fun n → .<fun x → spower n x>.
```

It looks like an eta-expansion, isn't it? The type checker quickly points the places with missing staging annotations. Here is the final version:

```
let spowern : int → (int → int) code =
   fun n → .<fun x → .~(spower n .<x>.)>.;;
⇝ val spowern : int → (int → int) code = <fun>
```

the desired specialized power. Let us check:

```
spowern 7;;
⇝ − : (int → int) code = .<
fun x_1 → x_1 ∗ ((∗ CSP square ∗) (x_1 ∗ ((∗ CSP square ∗) (x_1 ∗ 1))))>.
```

We have already seen that the code values can be printed. It turns out that even the code of functions can be printed, so we can always see what we have generated.

We also notice the mysterious *(∗ CSP square ∗)*. CSP stands for *cross-stage persistence*. Recall that square is a function that is defined and usable now. Yet we put it inside the brackets, assuming that it will also be available in the future. One may think of *(∗ CSP . . . ∗)* as a sort of a container that saves the present-stage value for future use.

Exercise 6. Is multiplication also a CSP?

One may safely assume that the standard library (which is available now) will also be available when running any generated code. Therefore, standard library CSP values need no containers. It is straightforward to turn square into a 'standard library' function: place its code into a separate file. Let's do that: create the file `square.ml` with one line

```
let square x = x ∗ x
```

in it. Compile it as ocamlc −c square.ml. Next re-define spower, making the just compiled square available (notice that the name of the module to **open** is the file name without the .ml suffix and with the first letter capitalized: see the OCaml reference manual for details)[4] :

```
open Square;;
#load "square.cmo";;

let rec spower : int → int code → int code = fun n x →
  if n = 0 then .<1>.
  else if n mod 2 = 0 then .< square .~(spower (n/2) x) >.
  else .<.~x ∗ .~(spower (n−1) x)>.
;;
⇝ val spower : int → int code → int code = <fun>
```

[4] open Square is an OCaml declaration that makes the interface of square.ml, i.e., the name square and its type known to the type checker. On the other hand, #load "square.cmo" is a top-level–specific dynamic linking directive, which makes the square's code available. The difference between open and #load is hence, in C terms, akin to the difference between #include of a library interface and linking in the library code.

```
let spowern : int → (int → int) code =
  fun n → .<fun x → ~(spower n .<x>.)>.;;
⇝ val spowern : int → (int → int) code = <fun>
let spower7c = spowern 7
⇝ − : (int → int) code = .<
fun x_2 → x_2 ∗ (Square.square (x_2 ∗ (Square.square (x_2 ∗ 1))))>.
```

Exercise 7. The specialized code looks quite optimal, except for the multiplication by one. How do you think we could eliminate that useless operation?

The variables in the printed spower7c code stand out: MetaOCaml systematically renames all bound variables, by appending the numerical suffix to a user-given name. To understand the reason, consider the following code and its generated code value

```
.<fun x → ~(let body = .<x>. in .<fun x → ~body>.)>.
⇝ .<fun x_1 → fun x_2 → x_1>.
```

If we rename the outer variable to y:

```
.<fun y → ~(let body = .<y>. in .<fun x → ~body>.)>.
⇝ .<fun y_1 → fun x_2 → y_1>.
```

the generated code denotes the same function as before (the K combinator). Such invariance with respect to systematic renaming of bound variables (α-conversion) is called hygiene.

Exercise 8. If MetaOCaml, like Lisp, did not rename bound variables, what code would have been generated for the two examples above? Would it denote the same function?

2.3 Offline code generation

We have specialized the power function: we have written the generator spowern that produces the (rather optimal) code for the function that takes x and returns x^n, for some fixed n. We can print the generated code. We should just as easily be able to save the generated code into a file. We can indeed. MetaOCaml even provides a special function for that: Print_code.format_code writes a code value into the supplied

output stream (technically, Format.formatter)[5]. The file with the saved spower7c code looks like any other OCaml code file: it can be compiled with bytecode- or optimizing OCaml compilers and linked into a user program.

We have just seen one way of using MetaOCaml: generating specialized code and whole libraries of specialized functions, for later linking. The specialized code can also be executed now.

2.4 Runtime specialization and its benchmark

To test the generated code, we can save it into a file as just described, compile and link with a test driver. Fortunately, there is an easier way. The generated code can be run right away: the MetaOCaml function Runcode.run : α code $\rightarrow \alpha$[6] takes a code value, compiles it, links it back into a program, and evaluates, returning the result. It truly invokes the compiler, from within a running program:

```
open Runcode;;
let spower7 = run spower7c;;
⇝ val spower7 : int → int = <fun>
```

We can now do a test:

```
spower7 2;;
⇝ − : int = 128
```

The specialized spower7 has the same type as the partially applied power7 we have seen earlier (repeated below for reference):

```
let power7 = power 7;;
⇝ val power7 : int → int = <fun>
```

Both return the 7th-power of their argument. However, power7 2 is power 7 2 in disguise, which recursively checks the parity of the first argument. In contrast, the specialized spower7c has no recursion (as seen from its code). All operations on n have been done when generating spower7c; the generated code merely multiplies x.

[5]Incidentally, this function will check that the code to write is really closed and hence can be successfully compiled. The absence of type errors is ensured by the MetaOCaml type system; the absence of unbound variables is checked at run-time of the *generator*: see Kiselyov (2014) for more discussion.

[6]Type variables in OCaml are written with the leading apostrophe: `'a`, `'b`, `'c`, etc. In typeset code, they are usually represented as Greek letters: α, β, γ, etc.

How much does the specialization help? Let's find out, by benchmarking power7 against spower7. For the sake of performance we will be using the optimizing (Meta)OCaml compiler rather than the bytecode-based metaocaml interpreter. In a new file `power_rts.ml` we place the earlier power, spower and spowern code, along with the timing code

```
let perf : (unit → α) → α = fun th →
  let start_time = Sys.time () in
  let r = th () in
  let elapsed_time = Sys.time () −. start_time in
  Printf.printf "\nit took %g secs\n" elapsed_time;
  r
```

and the actual benchmark, to compute 2^{20} 100000000 times (the Runnative module invokes the optimizing compiler for the specialized code):

```
let _ =
  let go count n x =
    let power_fn = power n in
    Printf.printf "Unspecialized power %d ^ %d is %d\n"
      x n (perf (fun () → for i = 1 to count do ignore (power_fn x) done;
                         power_fn x));
    Printf.printf "\nGenerating and compiling the specialized code (100 times)";
    let spower_fn = perf (fun () →
      for i = 1 to 99 do ignore(Runnative.run (spowern n)) done;
      Runnative.run (spowern n)) in
    Printf.printf "Specialized power %d ^ %d is %d\n"
      x n (perf (fun () → for i = 1 to count do ignore (spower_fn x) done;
                         spower_fn x))
  in
  go 100000000 20 2
```

Compiling the benchmark and running it

```
metaocamlopt −o power_rts square.ml power_rts.ml
./power_rts
```

gives the following results (on intel Core i3, 1.80GHz):

```
it took 3.842 secs
Unspecialized power 2 ^ 20 is 1048576

Generating and compiling the specialized code (100 times)
it took 0.087 secs

it took 0.757 secs
Specialized power 2 ^ 20 is 1048576
```

Specialization has been worth it, improving the performance more than four times. The improvement should be balanced against the time it took to generate the specialized code and then compile it. In our simplistic example, if the specialized function is invoked at least 28 200 times (= 87000 / (3.842 − 0.757)), the run-time specialization is worthwhile. One has to estimate such break-even point case-by-case.

Exercise 9. Write a power function of the type int → float → float, specialize it to the known int exponent, and benchmark the performance improvement.

Exercise 10 (Advanced)**.** The specialized spower7c code clearly invokes the external function square. Is there a way to inline it? Generate the specialized power that calls no extra functions and benchmark it, quantifying the improvement.

2.5 Recap

Two insights are worth stressing. First, a code value may represent open code, with free variables. For example, in .<**fun** x → .~(spower n .<x>.)>. the code value .<x>. is the future-stage variable x itself. We may store such variables in reference cells and pass them as arguments and function results. Staging hence lets us manipulate (future-stage) variables *symbolically*. However, we cannot determine the real name of a future-stage variable let alone tamper with it. A code value cannot be taken apart and examined. This pure generativity of MetaOCaml helps maintain hygiene: open code can be manipulated but the lexical scoping is still preserved.

The second insight is two different ways of using MetaOCaml. First is run-time specialization: specializing a frequently used function to data obtained at run-time, e.g., from user input. The specialized code can be immediately used, that is, run. Second, the generated code can be saved into a file. Since it is ordinary OCaml, it can be compiled with ordinary OCaml compilers and later linked into various *ordinary* OCaml applications. In this approach, the MetaOCaml facility to run code values is not essential: running is used for testing or for empirical code search.

Thus, MetaOCaml can be used not only for run-time specialization, but also for offline generation of specialized library code, e.g., of BLAS and Linpack libraries. One can benchmark the generated code and pick the version that performs the best on a given platform. MetaOCaml hence can be used just as ATLAS and other such metaprogramming systems. Later sections in the tutorial will stress this point.

Finally, our use of the type checker to tell which staging annotations should be put where makes one wonder if this process can be automated. Can we supply just the desired signature and obtain the specialized code, with correct annotations? Such type-driven binding-time analysis (BTA) does indeed work in simple cases (Asai, 2016). However, we shall soon see that often creative steps are required §3.1.

So far, we have been generating mutation-free, pure functional (applicative) code in the pure functional (applicative) style. In §3 we encounter generators that use mutable cells, as the natural outcome of specializing imperative code[7]. Later, §4 shows (pure functional, as it turns out) generators that produce imperative, high-performance numeric code.

2.6 A historical aside

The power example was the running example in the paper that begat partial evaluation as the research area: A.P. Ershov's 1977 paper (Ershov, 1977a). That example, in Figure 1 of the paper, can be seen on the following manuscript scan `http://ershov.iis.nsk.su/files/archive/fold0241/241_75.gif`. One sometimes comes across the linear-time version where x^n is computed by repeated multiplication by x. That was not the original example: Ershov, as most Russian programmers at that time were obsessed about efficiency. They would never consider an inefficient algorithm.

That well-known Ershov 1977 paper was the English version of

[7]Imperative generators are also more expressive – and dangerous, liable to produce the code with unbound variables. As we have mentioned earlier, MetaOCaml employs a code-generation–time check to detect such problem, as explained in Kiselyov (2014). Alerting to this problem even before running the generator, at type-checking time, is the subject of active current research (Kiselyov et al., 2016).

Ershov (1977b) that appeared in Doklady AN SSSR (Proceedings of the Academy of Sciences of USSR). For space reasons, the power example was omitted from the Doklady AN SSSR paper, but it can be seen in the notes to the November 1976 draft `http://ershov.iis.nsk.su/files/archive/fold0241/241_17.gif`. This is perhaps the first occurrence of the power function specialization.

For other drafts of this paper, see "The Ershov Archive for the history of computing"[8]

[8] `http://ershov.iis.nsk.su/archive/eaindex.asp?lang=2&gid=1480`

3

Filtering

Filters are ubiquitous in radio and all manner of speech and sound applications. When we want to listen to a radio station 90.5 MHz, from the incoming radio signals we filter out everything but the small band centered around 90.5 MHz. Two-dimensional filtering is likewise ubiquitous in cameras, printers and general image processing. Filters are often analogue. With specialized digital signal processors and now fast general-purpose CPU, it became possible to filter signals digitally, in software. A lucid, enjoyable introduction to digital filters is Richard Hamming's classical book Hamming (1997, 1983, 1989). Another helpful, online reference is Smith (2007).

This section transits from the introduction to real-life applications of staging. Our digital filters here are still not as optimal as they should be. We delay further optimization steps until we learn the systematic approach, in §4 and §5.

As before, we will be writing code in interaction with MetaOCaml, submitting (tentative) definitions to the MetaOCaml interpreter, examining its response, and testing the defined functions and generators right away. Let's invoke metaocaml and start a new session. (The complete code for this section is in the accompanying file `filter.ml`.)

Digital filters transform digitized signals represented as sequences of samples, or uniformly sampled signal values: x_i represents the signal amplitude at the time of the i-th sample. This tutorial deals with so-called finite impulse response (FIR) filters, of the following general form

$$y_i = \sum_{k=0}^{m-1} b_k \; x_{i-k}$$

The sample of the output signal y_i is a weighted sum of the m most recent input signal samples $x_i, x_{i-1}, \ldots x_{i-m+1}$. The weights b_k are the filter coefficients. The number $m-1$ is called the order of the filter. Since the number of coefficients (and often their values) are usually known, it makes sense to specialize filter computations. Speed is important: filters often process long signals or work online.

Here we encounter the generality-performance trade-off that we talked so much before, and which we shall encounter many more times. We would like to write the filter code once and for all, in full generality, and, hopefully, in a familiar to domain experts form. We would then like to specialize or optimize that code taking advantage of particular circumstances (e.g., knowledge of the coefficients). We do not attain the ideal in this section. Here, the specialization is by copying the code and manually inserting annotations (albeit with the help of the type checker). In §4 and especially §5 we shall see how to specialize code without rewriting and fat-fingering.

The filter equation above can be idiomatically encoded in OCaml as follows:

```
let filter : float array → float array → float array = fun b x →
  let m = Array.length b in
  let y i =
    if i < m−1 then x.(i) else
    let sum = ref 0.0 in
    for k = 0 to m−1 do
      sum := !sum +. b.(k) ∗. x.(i−k)
    done;
    !sum
  in
  Array.init (Array.length x) y        (∗ essentially, for−loop ∗)
```

The function filter takes the array of coefficients b and the array of input samples x, returning the (freshly allocated) array of samples of

the filtered signal. In OCaml, taking the i-th element of an array x is written as x.(i). For simplicity, we do not consider inplace or stream-like filtering. However, the specialization techniques we learn in this section apply as they are to streams. The core of filter is the function y that computes the i-th sample of the output signal. It is this function that we will be specializing (this is why our specializations will apply to streaming filters). The function Array.init: int $\rightarrow$ (int $\rightarrow \alpha$) $\rightarrow \alpha$ array in the last line of the code tabulates the function y: builds the array of the results of its invocations [| y 0; y 1; ... |]. The imperative style of y's code – with the mutable accumulator sum that is dereferenced (written as !sum in OCaml) and destructively updated by the := operation within the loop – is hard to miss. Such imperative OCaml code is (intentionally) patterned after C and Fortran so common in high-performance computing.

At high-level, y i computes the dot-product of the coefficient vector b with the length-m window into the input signal [| x[i]; x[i−1]; ... x[i−m+1] |]. In the later sections we shall see how to write the filter code at such high level.

Exercise 11. What to do about the boundary samples, when $i < m-1$? There are several options:

- $y_i = x_i$, when $i < m-1$
- $y_i = 0$, when $i < m-1$
- $y_i = x_o$, when $i < m-1$
- apply a smoothing window (e.g., Hamming window)

Write code for one of these other options for boundary processing.

The following functions creates sample input for testing purposes, of size n with two spikes (impulses) at locations i and j (which may be the same)

```
let impulses : int → int → int → float array = fun i j n →
  let x = Array.make n 0.0 in
  x.(i) ← 1.0;
  x.(j) ← 1.0;
  x
```

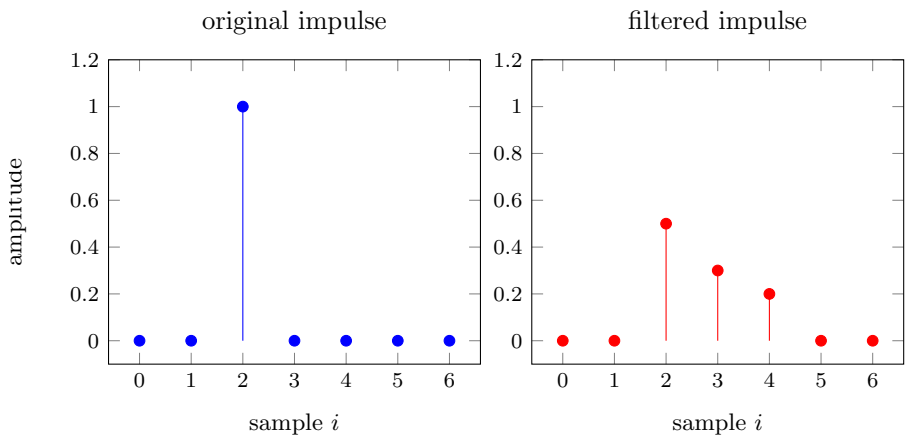

Figure 3.1: Original and filtered spike

Figure 3.1 contrasts the signal impulses 2 2 7 (the 7-sample signal with the single spike in the middle) with the filtered filter [| 0.5; 0.3; 0.2 |] (impulses 2 2 7) signal. In the output signal the spike is smoothed, as expected for the low-pass filter.

Exercise 12. How can you tell that the filter [| 0.5; 0.3; 0.2 |] is low-pass, that is, retains low frequencies and attenuates high frequencies?

The filtered impulse in Figure 3.1 looks like the filter coefficients b: that is why the coefficients are also called the *impulse response* of the filter.

3.1 Specializing to the known filter order

When designing a filter, one normally first decides on its order (the number of coefficients minus one) and then determines the coefficients to obtain the filter with the desired behavior, e.g., retaining a particular range of frequencies. The filter performance (operation count, required local memory) depends on its order; that is why the order is chosen first.

Let us specialize the earlier filter to the known order, that is, the length of the array b. We immediately run into a puzzle:

Exercise 13. A float array is completely known now: we can take its elements and use them. A float array code is known only in the future. What should be the type of the array with the known length; whose elements may be taken now but operated on later?

Having answered the exercise helps see the meaning and the utility of the following type abbreviations

```
type spine_arr =  float code array
type dyn_arr   =  float array code
```

We shall write the specialized filter code using the type-driven process described in §2.2. First we have to decide on the signature. The specialized filter_spine will take spine_arr as coefficients and generate the code that accepts the input signal and produces the output signal. Hence the first attempt:

```
let filter_spine : spine_arr → (float array → float array) code =
  fun b → .<fun x →
  ~(let m =  Array.length b in
  .<let y i =
    if i < m−1 then x.(i) else
    let sum =  ref 0.0 in
    for k =  0 to m−1 do
      sum :=  !sum +. b.(k) ∗. x.(i−k)
    done;
    !sum
  in
  Array.init (Array.length x) y
 >.)>.
```

We have copied-and-pasted the filter code, changing the name and the type signature and putting the most obvious staging annotations: the signature tells that the body has to be of the form **fun** b → .<**fun** x → . . . >. (Do you see that?) The length of b is known now, and is hence determined outside the brackets.

Although the code is far from finished, we submit it to the type checker anyway, to get a hint as to what to do next:

```
⇝                Characters 238−243:
      sum :=  !sum +. b.(k) ∗. x.(i−k)
```

Error: This expression has type float code
but an expression was expected of type float

When we see that an expression of `t code` was found but something of type `t` was expected, we know right away that an escape is missing[1]. Let's insert it (Again, we highlight the changes):

```
let filter_spine : spine_arr → (float array → float array) code =
  fun b → .<fun x →
  .~(let m = Array.length b in
  .<let y i =
     if i < m−1 then x.(i) else
     let sum = ref 0.0 in
     for k = 0 to m−1 do
       sum := !sum +. .~(b.(k)) ∗. x.(i−k)
     done;
     !sum
    in
    Array.init (Array.length x) y
  >.)>.
```

```
⇝                    Characters 244−245:
      sum := !sum +. .~(b.(k)) ∗. x.(i−k)
```

Error: A variable that was bound within brackets is used outside brackets
for example: .<fun x → .~(foo x)>.
Hint: enclose the variable in brackets, as in: .<fun x → .~(foo .<x>.)>.

The informative error message points out that the loop variable `k` is bound within the brackets and is not, hence, available now. Accessing the elements of `b` now is, however, the point of specialization. Therefore, `k` has to be turned into a present-stage variable – meaning that the entire loop has to be present-stage, like in the following:

```
let filter_spine : spine_arr → (float array → float array) code =
  fun b → .<fun x →
  .~(let m = Array.length b in
  .<let y i =
    if i < m−1 then x.(i) else
    let sum = ref 0.0 in
  .~(for k = 0 to m−1 do
      sum := !sum +. b.(k) ∗. .<x.(i−k)>.
    done;)
    !sum
   in
   Array.init (Array.length x) y
  >.)>.
⇝                    Characters 223−226:
      sum := !sum +. b.(k) ∗. .<x.(i−k)>.
```

Error: A variable that was bound within brackets is used outside brackets

[1] §3.3 shows a different way to deal with this error.

The type checker suggests that sum also has to be bound at the present stage. The moment of thought shows that the type of sum has to change too. Bound within brackets, sum had the type float ref. When bound outside brackets, it has to be something like float ref code, a future-stage reference cell. However, in that case we cannot dereference sum within the loop now. The only other choice is to make sum of the type float code ref: the present-stage reference cell, whose contents is a code value.

```
let filter_spine : spine_arr → (float array → float array) code =
  fun b → .<fun x →
  .~(let m = Array.length b in
  .<let y i =
    if i < m−1 then x.(i) else
  .~(let sum = ref .<0.0>. in
    for k = 0 to m−1 do
      sum := !sum +. b.(k) ∗. .<x.(i−k)>.
    done;
    !sum)
   in
   Array.init (Array.length x) y
 >.)>.
```

```
⇝                    Characters 234−238:
      sum := !sum +. b.(k) ∗. .<x.(i−k)>.
```

Error: This expression has type float code
but an expression was expected of type float

We are getting somewhere, but not there yet. As we have learned, the error message says that !sum has to be escaped. Since an escape may only appear within brackets, we have to insert the enclosing brackets as well:

```
let filter_spine : spine_arr → (float array → float array) code =
  fun b → .<fun x →
  .~(let m = Array.length b in
  .<let y i =
    if i < m−1 then x.(i) else
  .~(let sum = ref .<0.0>. in
    for k = 0 to m−1 do
      sum := .< .~(!sum) +. .~(b.(k)) ∗. x.(i−k) >.
    done;
    !sum)
   in
   Array.init (Array.length x) y
 >.)>.
⇝ val filter_spine : spine_arr → (float array → float array) code = <fun>
```

And it worked! Let us see the code: take the filter order to be two and hence apply filter_spine to an array of three code values:

```
.<fun b1 b2 b3 → .~(filter_spine [|  .<b1>.; .<b2>.; .<b3>.  |])>.;;
⇝ − : (float → float → float → float array → float array) code =  .<
fun b1_1  → fun b2_2  → fun b3_3  →
  fun x_4  →
    let y_6 i_5 =
      if i_5 < (3 − 1)
      then x_4.(i_5)
      else
        ((0.0 +. (b1_1 ∗. (x_4.(i_5 − 0)))) +. (b2_2 ∗. (x_4.(i_5 − 1))))
          +. (b3_3 ∗. (x_4.(i_5 − 2)))
       in
    Array.init (Array.length x_4) y_6>.
```

This is not bad at all: even with imperfections (like additions to zero) the code looks rather close to the ideal. It was perhaps unexpected that specializing to the statically known b array length caused the loop over b to be unrolled. The type-directed annotation-insertion has been more complicated than in the power example of §2.2, requiring a bit of creativity.

The original code was mutation-laden; the mutations are all preserved in filter_spine. They are executed at the present stage: filter_spine is hence an imperative generator, accumulating (open!) code fragments in mutable cells. The generated code is, however, purely applicative, with the accumulation done earlier. In contrast, §4 uses a pure functional generator to produce imperative code.

3.2 Specialization to the known coefficients

Once the filter is designed, its coefficients (the impulse response) are known. The production filter code should take advantage of this knowledge. Let us therefore specialize filter_spine further: not only to the number of coefficients but also their values.

Once again we use the type-driven annotation-insertion process. To begin, we take the filter_spine code, change the name to filter_staged and attach the new signature. Even though we have not adjusted the body, we submit the code to the type checker anyway:

```
let filter_staged : float array → (float array → float array) code =
  fun b → .<fun x →
```

```
  .~(let m = Array.length b in
  .<let y i =
    if i < m-1 then x.(i) else
  .~(let sum = ref .<0.0>. in
    for k = 0 to m-1 do
      sum := .< .~(!sum) +. .~(b.(k)) *. x.(i-k) >.
    done;
    !sum)
   in
   Array.init (Array.length x) y
 >.)>.
⇝                    Characters 255-262:
      sum := .< .~(!sum) +. .~(b.(k)) *. x.(i-k) >.
```

Error: This expression has type float but an expression was expected of type float code

One may have expected the complaint about b.(k): since the coefficients are known now, the escape is no longer appropriate. We remove it:

```
let filter_staged : float array → (float array → float array) code =
  fun b → .<fun x →
  .~(let m = Array.length b in
  .<let y i =
    if i < m-1 then x.(i) else
  .~(let sum = ref .<0.0>. in
    for k = 0 to m-1 do
      sum := .< .~(!sum) +. b.(k) *. x.(i-k) >.
    done;
    !sum)
  in
  Array.init (Array.length x) y
 >.)>.
;;
⇝ val filter_staged : float array → (float array → float array) code = <fun>
```

and, all of the sudden, obtain the working generator. Has it really been that simple? Let us try out on sample coefficients:

```
filter_staged [| 0.5; 0.3; 0.2 |];;
⇝ - : (float array → float array) code = .<
fun x_7 →
  let y_9 i_8 =
    if i_8 < (3 - 1)
    then x_7.(i_8)
    else
      ((0.0 +. ((* CSP b *).(0) *. x_7.(i_8 - 0))) +.
         ((* CSP b *).(1) *. x_7.(i_8 - 1)))
        +. ((* CSP b *).(2) *. x_7.(i_8 - 2))
     in
  Array.init (Array.length x_7) y_9>.
```

Albeit technically correct, it is not quite what we wanted: Although both the array b and the indices are known now, we nevertheless access its elements later (and so have to transport b from the present to the future stage: hence CSP).

Exercise 14. Before we discuss the solution, any thoughts on how to solve the problem?

Since b.(k) appears inside the brackets, the element-access operation ("the dot") is performed only when the generated code is run. Yet it can be done now since both of its arguments are known. The generated code should contain the result of the array access but not the operation itself. Let-bindings come to mind: **let** x = expr **in** body also evaluates (in a call-by-value language) expr ahead of body, with only the result of expr being used within the body. Our case calls for the evaluation of b.(k) not just earlier but in the earlier stage, which can be arranged by placing escapes (to move to the earlier stage) and brackets (to shift the result back into the future) accordingly:

```
let filter_staged : float array → (float array → float array) code =
  fun b → .<fun x →
  .~(let m = Array.length b in
  .<let y i =
    if i < m−1 then x.(i) else
    .~(let sum = ref .<0.0>. in
      for k = 0 to m−1 do
        sum := .< .~(!sum) +. .~(let bk = b.(k) in .<bk>.) ∗. x.(i−k) >.
      done;
    !sum)
  in
  Array.init (Array.length x) y
 >.)>.
;;
⇝ val filter_staged : float array → (float array → float array) code = <fun>
```

The let-expression may also be pulled out of escape:

```
let filter_staged : float array → (float array → float array) code =
  fun b → .<fun x →
  .~(let m = Array.length b in
  .<let y i =
    if i < m−1 then x.(i) else
  .~(let sum = ref .<0.0>. in
    for k = 0 to m−1 do
     let bk = b.(k) in
      sum := .< .~(!sum) +. bk ∗. x.(i−k) >.
```

```
    done;
    !sum)
  in
  Array.init (Array.length x) y
 >.)>.
;;
⇝ val filter_staged : float array → (float array → float array) code = <fun>
```

After this small change, the generated filter code

```
filter_staged [|  0.5; 0.3; 0.2  |];;
⇝ − : (float array → float array) code = .<
fun x_10  →
  let y_12 i_11 =
    if i_11 < (3 − 1)
    then x_10.(i_11)
    else
      ((0.0 +. (0.5 ∗. x_10.(i_11 − 0))) +. (0.3 ∗. x_10.(i_11 − 1))) +.
        (0.2 ∗. x_10.(i_11 − 2))
     in
  Array.init (Array.length x_10) y_12>.
```

looks rather close to the ideal.

For a bit of fun at the end, try the code near the end of the file `filter.ml` that accompanies the tutorial. The code builds sample waveforms (including a siren) and plays them. Apply various filters and hear the difference!

3.2.1 A Digression on CSP

The code we have just generated for the staged filter is, in fact, closer to the ideal than one might have expected. Recall, the key was the let-expression **let** bk = b.(k) **in** .<bk>. that obtained the k-th element of the array b at the present stage, and shifted that value bk to the future, by putting it inside the brackets. That means that bk is a CSP. How come we did not see any *(∗ CSP ∗)* in the generated code (but we saw plenty of them in the earlier, unsatisfactory attempt)?

The CSP story is quite complicated[2]. In §2.2 we have talked about standard library functions like ∗ (or, in this section, +. and ∗.). The standard library is expected to be always available, now and when when the generated code is later compiled and run. Therefore, the generated code may refer to these functions merely by their names.

[2] told in some detail in `http://okmij.org/ftp/ML/MetaOCaml.html#CSP`

On the other hand, when a local present-stage identifier (e.g., a function argument) appears inside the brackets, the generated code cannot refer to it by name: the local scope in which the identifier is bound will likely exit well before the generated code is run. Here we have to transport the value bound to the identifier from the present to the future stage. To be precise, given a value we have to build code that will evaluate to that value. This generally undecidable 'unevaluation' is what makes CSP so complex.

There are simple cases however: values for which the language provides literal notation, e.g., numbers, strings, booleans. For example, the code that evaluates to the machine integer 10 is the literal 10. MetaOCaml builds such literals automatically:

```
.<10>.
⇝ − : int code =  .<10>.

let n =  10 in .<n>.
⇝ − : int code =  .<10>.
```

Both expressions generate the code with the literal 10. In the case of .<10>. we have entered that literal ourselves. In the second expression, that literal is built by MetaOCaml when the generator is run. Floating-point and other numeric literals are handled similarly.

The other cases of CSP are complicated. For function values, producing code that evaluates to (behaviorally) the same function is generally undecidable. MetaOCaml employs various tricks, for example, building a reference to the object in the generator heap. It is this heap pointer that is printed as *(∗ CSP ∗)*. Such a heap pointer is exactly what we want for CSP when doing run-time code specialization and linking the specialized code back into the running program. When generating code for a later, offline use, the trick does not work at all: by the time the code is used the generator will be long gone along with its heap.

Historically, MetaOCaml emphasized run-time code specialization, encouraging users not to mind CSP and let MetaOCaml work its magic. In this tutorial we want to promote less reliance on magic. Let us try to be more mindful about CSP; in particular, let's use the operations in the module `lift.ml` such as Lift_list(Lift_float.lift) to explicitly convert

a present-stage value to the future-stage code that will evaluate to it. We will see the examples in later sections.

3.3 Smarter specialization

The two earlier specializations, §3.1 and §3.2, completely unrolled the loop that computed the output sample as the weighted sum of the input samples within the filter window. The unrolling removes the overhead of the iteration, at the expense of the code size. In a filter of high order, with many coefficients, the fully unrolled loop may take too much code and hinder the performance. Keeping the loop as it is a better trade-off then.

Looking back to §3.1, we now develop a 'smarter' filter specialized to the known filter order. If the order is small enough (that is, smaller than a threshold), the weighted sum loop is unrolled. The specialized filter_spine from §3.1 will see to it. Otherwise, the loop will be kept as it was. Hence, in the first approximation, the smarter filter_spine is as follows:

```
let filter_spine_adaptive ?(threshold= 3) :
    spine_arr → (float array → float array) code =  fun b →
 let m =  Array.length b in
 if m ≤ threshold then
   filter_spine b
 else
   .<fun x →
     let y i =
      if i < m−1 then x.(i) else
      let sum =  ref 0.0 in
      for k =  0 to m−1 do
        sum :=  !sum +. b.(k) ∗. x.(i−k)
      done;
     !sum
    in
    Array.init (Array.length x) y>.
```

Although we try to stay away from 'unusual' OCaml features, default arguments (notated as ?(threshold= 3)) are too good to pass on. Technically, filter_spine_adaptive has two arguments, the threshold and the array of coefficients. If the function is invoked as filter_spine_adaptive b, then threshold takes the default value 3. The default is explicitly overridden in the invocation filter_spine_adaptive ~threshold:2 b.

The code however is not accepted by the type-checker:

```
⇝                    Characters 328–333:
        sum := !sum +. b.(k) *. x.(i-k)
Error: This expression has type float code
       but an expression was expected of type float
```

We have seen that error before, in §3.1. There, we put the escape around b.(k), which, in turn, led to escaping, hence unrolling, of the whole loop. The problem can be fixed in a different way, however. Recall, b is a spine_arr, that is, float code array. If it were a dyn_arr, that is, float array code, we could have spliced it into the loop, as (~b).(k), giving us the future-stage float expression expected by the type checker. The problem hence is converting float code array to float array code.

Exercise 15. In §2.2 we turned spower: int code → int code into spowern of the type (int→int) code. The conversion was a sort of eta-expansion. Taking this as a hint, try to develop the float code array to float array code conversion on your own, before reading further. What other staged eta-expansions could be useful?

Exercise 16. As another hint to the previous exercise, try to write the float code list → float list code conversion first. Do you see a connection with CSP – converting a list to a code that will evaluate to that list?

The conversion looks concise, albeit probably not easy to grasp at first. The key idea is to convert an array to a list, and then back to an array (but at the future stage). Since list is an inductive data structure, it is straightforward to build the code of lists (see Exercise 16).

```
let dyn_arr_of_spine_arr : spine_arr → dyn_arr = fun xa →
  .<Array.of_list
    .~(Array.fold_right (fun x acc → .<.~x :: .~acc>.) xa .<[]>.)>.
```

To test,

```
dyn_arr_of_spine_arr [| .<1.>.; .<2.>.; .<3.>. |]
⇝ - : dyn_arr = .<Array.of_list [1.; 2.; 3.]>.
```

The adaptive specialized filter code is now complete:

```
let filter_spine_adaptive ?(threshold= 3) :
    spine_arr → (float array → float array) code = fun b →
  let m = Array.length b in
```

```
if m ≤ threshold then
  filter_spine b
else
  .<let bdyn = ~(dyn_arr_of_spine_arr b) in
   fun x →
    let y i =
     if i < m−1 then x.(i) else
     let sum = ref 0.0 in
     for k = 0 to m−1 do
       sum := !sum +. bdyn.(k) ∗. x.(i−k)
     done;
    !sum
  in
  Array.init (Array.length x) y>.
⇝ val filter_spine_adaptive :
   ?threshold:int → spine_arr → (float array → float array) code = <fun>
```

We let-bound the converted array of coefficients, so that the memory for the future-stage array is allocated outside the loop – even outside the function y, rather than on each iteration through the loop. We can easily see that in the generated code (given high-enough order: 3 in our case)

```
.<fun b1 b2 b3 b4 →
        ~(filter_spine_adaptive [| .<b1>.; .<b2>.; .<b3>.; .<b4>. |])>.
⇝ − : (float → float → float → float → float array → float array) code =
.<
fun b1_28 → fun b2_29 → fun b3_30 → fun b4_31 →
  let bdyn_32 = Array.of_list [b1_28; b2_29; b3_30; b4_31] in
  fun x_33 →
    let y_37 i_34 =
      if i_34 < (4 − 1)
      then x_33.(i_34)
      else
        (let sum_35 = Pervasives.ref 0.0 in
         for k_36 = 0 to 4 − 1 do
           sum_35 :=
             ((! sum_35) +. ((bdyn_32.(k_36)) ∗. (x_33.(i_34 − k_36))))
         done;
         ! sum_35)
       in
    Array.init (Array.length x_33) y_37>.
```

We have achieved the desired result. Yet a slight dissatisfaction remains. First, we had to effectively write the filter code twice, for the different levels of specializations. Second, we were lucky in the foresight of let-binding the array of coefficients just at the right place. It was clearly worthwhile. It would have been more worthwhile to rely less on luck

and foresight, and having the let-binding appear 'automatically' where needed, and at the right place. Both wishes are attainable, in further sections. Read on!

3.4 Further challenges

The specialized filter code that we have just built is close to the ideal, but not quite. We notice useless operations i−0 and the addition to 0.0. There is a more significant problem, best seen in the echo filter:

$$y_i = \frac{1}{1+\alpha} x_i + \frac{\alpha}{(1+\alpha)} x_{i-P}$$

where the delay (corresponding to P) should be about 0.2 secs, and α is about 0.95. In the specialized echo code

```
let alpha = 0.95 in let alpha1 = 1. +. alpha in
 filter_staged [| 1. /. alpha1; 0.; 0.;0.;0.;0.; alpha /. alpha1 |];;
⇝ − : (float array → float array) code = .<
fun x_16 →
  let y_18 i_17 =
    if i_17 < (8 − 1)
    then x_16.(i_17)
    else
      (((((((0.0 +. (0.512820512821 ∗. (x_16.(i_17 − 0)))) +.
              (0. ∗. (x_16.(i_17 − 1))))
             +. (0. ∗. (x_16.(i_17 − 2))))
            +. (0. ∗. (x_16.(i_17 − 3))))
           +. (0. ∗. (x_16.(i_17 − 4))))
          +. (0. ∗. (x_16.(i_17 − 5))))
         +. (0. ∗. (x_16.(i_17 − 6))))
        +. (0.487179487179 ∗. (x_16.(i_17 − 7)))
     in
   Array.init (Array.length x_16) y_18>.
```

the multiplications by zero stand out. For echo to be audible, the delay should be at least 0.2 seconds, which translates to many – thousands – of zero multiplications.

Can we count on an optimizing compiler to remove floating-point multiplications by 0.0? What if the other multiplicand is inf or NaN? For this optimization to be applicable, one needs an a priori knowledge that the input signal samples are always proper floating-point numbers. As is often the case, effective optimizations depend on domain knowledge and hence are not usually supported by a general-purpose optimizing

compiler. That is why writing generators that incorporate the domain-specific, expert knowledge is so important.

Digital filters is a rich and enjoyable area with many interesting projects. One may further try:

- different boundary conditions and smoothing windows;
- generating code for filtering inplace;
- playing with the sound;
- implementing a design methodology for particular kinds of filters (low-pass, two-tap, etc.);
- writing a framework, graphical or otherwise, for filter design.

3.5 Recap

The filtering example taught us loop unrolling, and the role of let-insertion to move computations to an earlier stage. We saw for ourselves that effective optimizations often depend on domain knowledge. We also saw that MetaOCaml is powerful enough for the needed optimizations, but the techniques so far were ad hoc. We need a systematic design, building optimizations from separate components. We need DSL. The rest of the tutorial will build a DSL, for linear algebra.

4

Linear Algebra DSL: Complex Vector Arithmetic and Data Layout

We turn to the systematic approach to generating optimal code. Our goal is to write code in the clear, obviously correct way, in the notation close to the one used by domain experts – and then interpret that notation as a code generator, adding optimizations one-by-one. Each optimization embodies domain knowledge and is to be developed independently by domain experts. In contrast to the previous sections, we shall specialize and optimize code without having to keep re-writing and annotating it.

What makes optimizations modular and composable is a common domain-specific language (DSL) – or, to be precise, a particular DSL implementation technique: so-called tagless-final style (Kiselyov, 2012; Carette et al., 2009). When embedding a DSL in a host language such as OCaml, it is common to represent DSL expressions as values of an algebraic data type: each DSL language form such as numeric literal or addition corresponds to a data constructor of that data type. This is a so-called 'deep embedding'. The tagless-final approach is the alternative. It is defined as a mapping of DSL expressions to host language (OCaml) values, which hence serve as the meanings, or denotations, of the expressions. The mapping is compositional: each DSL expression

form corresponds to a host function that computes the meaning of that form from the meanings of its immediate components. The tagless-final approach hence represents a DSL as an algebra (in the abstract algebra sense), also called an 'algebraic structure'. One may choose various sets of OCaml values to serve as the meanings for DSL expressions. In other words, one may interpret the same expression in different ways, depending on the task at hand. A code generator is but one interpretation. DSLs in the tagless-final style are extensible: features can be added at wish, without breaking any of the earlier written code. The interpretations are extensible as well, with (domain-specific) optimizations added as needed. The DSL is typed. The interpretations – code generators – are, like MetaOCaml itself, typed. Therefore, the generated code shall always be well-formed, hygienic and well-typed. Types not only prevent clearly wrong generators; they will help writing them. Thus, in this and the following sections we will be developing DSLs embedded in (Meta)OCaml in the tagless-final style.

Our subject is Linear Algebra. This section deals with vector-vector operations – so-called BLAS Level 1 (BLAS) – and data layout. §5 continues to matrix-vector operations (BLAS Level 2), algebraic simplifications and domain-specific sparse matrix optimizations. Our DSL code will look like

```
let vmult  vout v1 v2 =  vout :=  v1 *. v2
let mvmult vout a v   =  vout :=  a * v
```

in the familiar to many Matlab-like notation – and yet generate familiar to some imperative loops specialized to particular data layouts. Incidentally, there is not a glimpse of brackets and escapes. Truly, with well-chosen abstractions the user-facing code (and many optimizations layers) have no overt staging annotations. They appear only in very low-level generators.

4.1 Data layout problem

This problem has been posed by the HPC researcher Reiji Suda from the University of Tokyo – as part of Shonan Challenges, by HPC researchers to meta-programmers (Aktemur et al., 2013). The problem is

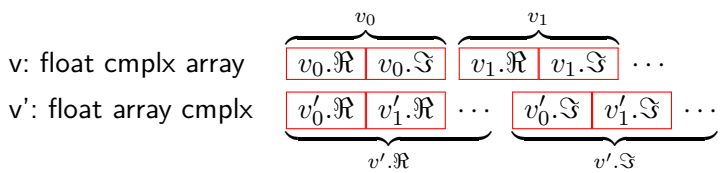

Figure 4.1: Data layouts: an array of structures v and a structure of arrays v'

a realistic specialization for a particular data representation. Some computer architectures favor so-called arrays-of-structures whereas GPUs and vector processors structures-of-arrays. Porting a program from GPU to a supercomputer or vice versa requires changing data representation. The goal is to automate such changes, to implement the algorithm only once and then mechanically adjust it to different data layouts.

Specifically, the problem is to implement point-wise multiplication of two complex vectors in such a way so to allow an easy specialization to different complex vector representations:

- float cmplx array
  an array whose elements are complex numbers: pairs of floating-point numbers for the $\Re$ and $\Im$ parts. This, so-called array-of-structures representation is popular in cache-based machines.

- separate float array ($\Re$ part) and float array ($\Im$ part)
  This structure-of-arrays representations is preferable for vector processors and GPU.

- a single float array: first half for the $\Re$ part, second for $\Im$ (a variant of the above).

The different layouts are schematically compared in Figure 4.1. To make the distinction even more apparent, let's compare two versions of the code that stores real 1 into the first element of a complex array. One version deals with float cmplx array and the other with the structure of arrays float array cmplx. (Although we are yet to introduce the cmplx data type precisely, the code should be self-explanatory):

```
let store_one_aos : float cmplx array → unit = fun v →
```

```
  v.(0) ← {re= 1.0; im= 0.0}

let store_one_soa : float array cmplx → unit =  fun v' →
  v'.re.(0)  ← 1.0;
  v'.im.(0) ← 0.0
```

We should be able to automatically generate different versions of the code for all these different representations. It should also be easy to add new (e.g., packed) representations.

4.2 Abstracting arithmetic

We begin the systematic approach by designing a DSL for things that can be added and multiplied. This may seem like an odd place to start, especially to an HPC practitioner: arithmetic is so fundamental in HPC, any overhead to its operations is intolerable. A cynic might regard our DSL as an excuse to indulge in abstract nonsense. We will see very soon however that our abstraction is both practical, and, in the end, overhead-free.

Once again we will be developing code in interaction with the MetaOCaml top-level, submitting definitions, examining MetaOCaml's response, testing the defined functions and generators right away. In preparation, let's invoke metaocaml and start a new session. (The complete code for this section is in the accompanying files `ring.ml`, `vector.ml`, `complex.ml`.)

We start with the minimal DSL, just with the operations needed at the moment. More can always be added later: extensibility is the strong point of the tagless-final approach. For now, we need a set of objects that we can add, subtract and multiply, with two distinguished elements zero and one. This collection of operations can be arranged in a so-called (module) *signature*:

```
module type RING =  sig
  type t
  val zero : t
  val one  : t
  val add  : t → t → t
  val sub  : t → t → t
  val mul  : t → t → t
end
```

The declaration **type** t gives the (very unimaginative) name to the type of our objects. The type is abstract: nothing is known about it except for its name, t. Concrete implementations will realize t as floating-point- or complex numbers, etc. The DSL code however cannot know or depend on the concrete realization of t (the OCaml type system will see to it) and hence remains applicable to all possible realizations. We could have chosen fancier names like +% or even + instead of add.

For now we merely define the names of the operations on t and their arities. The operations are implicitly assumed to satisfy the familiar properties: add is commutative and associative with the unit zero; mul is associative with the unit one and zero zero; mul distributes through add. In a word, we define a mathematical *ring*. We have not stated these algebraic laws anywhere, yet. We will come to them in §5.

One may view the signature RING as defining a mini-DSL with two constants and three operations. (In fact, this is the tagless-final standpoint). The DSL can be used right away:

```
module ExArith(R:RING) = struct
  open R
  let xsq1 x = add (mul x x) one
end
⇝ module ExArith : functor (R : RING) → sig val xsq1 : R.t → R.t end
```

e.g., to write the function to square its argument and add one. The first line says that the implementation of RING is abstracted away from the DSL expression xsq1 that follows. Technically speaking, a tagless-final DSL expression takes the form of a functor, parameterized by a RING implementation – *any* implementation. The lines except for **let** xsq1 x = ... line are the boilerplate, which hardly ever changes and may be automated (for example, as illustrated in Yallop (2016) using the technique of White et al. (2014)). We will abstain, however, from syntax sugar in this tutorial, keeping it broadly understandable and portable.

We should mention in passing that the (admittedly, ungainly) module notation is not strictly necessary. We may also write

```
let xsq1 (type a): (module RING with type t = a) → (a→a) =
  fun (module R) → let open R in
    fun x → add (mul x x) one
⇝ val xsq1 : (module RING with type t = α) → α → α = <fun>
```

where add (mul x x) one is an ordinary OCaml expression. We leave the explanations of the preceding boilerplate part as an exercise; incidentally, it, too, can be abbreviated. Thereafter, we will be using the older, more conventional, albeit ungainly, module notation.

To try out the just defined DSL function, we need an implementation of RING. Floating-point numbers (to a good approximation) form a ring[1]:

```
module RingFloat = struct
  type t = float
  let zero = 0.
  let one = 1.
  let add = Pervasives.( +. )
  let sub = Pervasives.( -. )
  let mul = Pervasives.( *. )
end
```

The first line after the module header tells that the abstract type t is being realized as the OCaml float. The operations are the standard float arithmetic. Supplying this implementation to ExArith gives the function xsq1 that squares its floating-point argument and adds one. We can try this function out, say, by applying it to 2.0:

```
let f = let module M = ExArith(RingFloat) in M.xsq1
 in f 2.0
⇝ - : RingFloat.t = 5.
```

The example in the alternate notation looks even simpler:

```
let f = xsq1 (module RingFloat)
 in f 2.0
⇝ - : RingFloat.t = 5.
```

There are many more implementations of RING, for example:

```
module RingFloatCode = struct
  type t = float code
  let zero = .<0.>.
  let one = .<1.>.
  let add = fun x y → .<~x +. ~y>.
  let sub = fun x y → .<~x -. ~y>.
  let mul = fun x y → .<~x *. ~y>.
end
```

[1]Enclosing an infix operation in parenthesis makes it a two-argument function: (+.) x y is hence equivalent to x +. y. In fact, (+.) is treated as any other identifier. In particular, any module may define its own version of the (+.) operation. To disambiguate, we qualify the identifier with the name of its defining module. Pervasives is the name of the most basic, initially opened OCaml module.

Here, add no longer adds floating-point numbers. It is now a generator of addition. Passing RingFloatCode to the very same DSL expression ExArith gives a different interpretation to xsq1. It is now a generator!

```
let f = let module M = ExArith(RingFloatCode) in M.xsq1
  in .<fun x → .~(f .<x>.) >.
⇝ − : (float → float) code = .<fun x_3 → (x_3 ∗. x_3) +. 1.>.
```

This is the payoff of the abstraction: "write once, run anywhere". Once defined, a DSL expression may be interpreted either as a present-stage computation (to run now, for testing) or as a code generator. The generated code .<**fun** x_3 → (x_3 ∗. x_3) +. 1.>. in the last example has no traces of modules, functors and other abstractions; it uses nothing but built-in OCaml floating-point operations and is hence as fast as you can make it in OCaml. As promised, the RING abstraction eventually has no overhead: truly the abstraction without guilt.

§5 shows more implementations of RING, doing simplifications based on the ring algebraic laws. For now, we turn to complex numbers – which, in one representation, are pairs of re and im parts:

```
type α cmplx = {re: α; im: α}
```

For technical reasons (so-called constructor restriction Kiselyov (2014)), this declaration should be placed into a separate file, say, `cmplx.mli` – to be compiled as ocamlc −c cmplx.mli and brought into the top-level as

```
open Cmplx
```

If the re and im parts of a complex number are in a ring, the complex number is itself in a ring. The following definition is the constructive proof:

```
module RingCmplx(R:RING) = struct
  type t = R.t cmplx
  let zero = {re = R.zero; im = R.zero}
  let one = {re = R.one; im = R.zero}
  let add = fun x y →
    {re = R.add x.re y.re;
     im = R.add x.im y.im}
  let sub = fun x y → ...
  let mul = fun x y → ...
end
```

Exercise 17. Implement the missing sub and mul operations of RingCmplx.

That means, RingCmplx(RingFloat) is a RING implementation, and can be passed to the very same running example ExArith giving us the square-increment function – this time, on the complex floating-point numbers:

```
module FloatComplex = RingCmplx(RingFloat)
type float_complex = FloatComplex.t

let f = let module M = ExArith(FloatComplex) in M.xsq1
  in f {re= 2.0; im= 1.0} (* test *)
⇝ – : FloatComplex.t = {re = 4.; im = 4.}
```

Since we will be constantly using complex floats, we define the FloatComplex abbreviation for the RingCmplx(RingFloat) implementation, and also float_complex for its type.

We also obtain the generator (generator template, to be precise) for xsq1 on complex floats, for free:

```
module FloatCodeComplex = RingCmplx(RingFloatCode)
type float_code_complex = FloatCodeComplex.t

let module M = ExArith(FloatCodeComplex) in M.xsq1
⇝ – : FloatCodeComplex.t → FloatCodeComplex.t = <fun>
```

Again we have introduced the abbreviations for the implementation module and the type.

To use the template conveniently, let's define

```
let of_complex_code : float cmplx code → float_code_complex =
  fun x → {re = .<~x.re>.; im = .<~x.im>.}
let of_code_complex : float_code_complex → float cmplx code =
  fun {re;im} → .<{re = ~re; im = ~im}>.
```

The code to square-increment a number – a complex number, this time – is then obtained as

```
let _ =
  let f = let module M = ExArith(FloatCodeComplex) in M.xsq1
  in .<fun x →
        ~(of_code_complex (f (of_complex_code .<x>.)))>.
⇝ – : (float Cmplx.cmplx → float Cmplx.cmplx) code = .<
fun x_3 →
  {Cmplx.im =
     (((x_3.re *. x_3.im) +. (x_3.im *. x_3.re)) +. 0.);
    Cmplx.re =
```

```
    (((x_3.re *. x_3.re) -. (x_3.im *. x_3.im)) +. 1.)
}>.
```

Exercise 18. Explain the purpose of of_complex_code and of_code_complex

Exercise 19. Write another implementation of RING, with t as float cmplx code. What are its advantages and disadvantages compared to FloatCodeComplex?

4.3 Abstracting vectors

The problem of specializing to various vector memory layouts calls for finding a suitably abstract representation of vectors. An ordinary floating-point array may be thought of as a function int $\rightarrow$ float over a range of integers[2]. Generalizing, we define (an input) vector as a function that takes an index and returns a value at that index; the vector also carries the minimal and the maximal indices for the range on which the vector is defined.

type (ι,α) vec = Vec **of** $\iota * (\iota \rightarrow \alpha)$

The index ι is presumed to be an enumerable type. For simplicity we assume the default value zero for the lower bound of the index range and carry only the upper bound: the length of the index range. This vec is sometimes called a 'pull vector'. Arrays are mutable data structures. To represent array assignments we introduce output vectors:

type (ι,α,ω) ovec = OVec **of** $\iota * (\iota \rightarrow \alpha \rightarrow \omega)$

as a pair of the vector length and the mutation function. The latter takes an index and a value to store at that index, and returns some representation of the updated vector. For a destructive assignment, that result ω may be unit.

To get a better feel for abstract vectors, let's see how an OCaml array can be represented, as a pair of an input and an output vector:

[2]FORTRAN used the same notation A(I) both for accessing an I-th element of an array and invoking a function on the argument I.

```
let vec_of_array : α array → ( (int,α) vec ∗ (int,α,unit) ovec ) = fun a →
 let n = Array.length a in
 ( Vec  (n, fun i   → a.(i)),
   OVec (n, fun i v → a.(i) ← v)
 )
```

OCaml arrays are mutated destructively, therefore, the result type of the mutation function is unit.

Element-wise operations on abstract vectors can be defined generically, once and for all. For example, the map operation (applying a user-supplied function to each element)

```
let vec_map : (α → β) → (ι,α) vec → (ι,β) vec =
 fun tf (Vec (n, f)) → Vec (n, fun i → tf (f i))
```

and its extension to two arguments

```
let zip_with : (α→β→γ) → (ι,α) vec → (ι,β) vec → (ι,γ) vec =
  fun tf (Vec (n1,f1)) (Vec (n2,f2)) →
    Vec (n1, fun i → tf (f1 i) (f2 i))
```

Exercise 20. We have assumed that the two argument vectors to zip have the same size. Try to either relax or assure this assumption.

Vector assignment is a variation of zip_with, when the first argument is an output vector

```
let vec_assign : (ι,α,ω) ovec → (ι,α) vec → (ι,ω) vec =
 fun (OVec (no,fo)) (Vec (ni,fi)) →
   Vec(no, fun i → fo i (fi i))
```

Recall, abstract vectors support both destructive and non-destructive assignment. The mutation function of the output vector returns a representation of the updated element. Thus the result of vec_assign is an input vector with the element assignment results.

The definition of vec_assign gives a hint that something else is needed to use vectors in practice: specifically, operations to iterate upon them. By themselves, vec and ovec describe uniform operations on vector elements: mapping. We also need reduce.

The operations on vectors as a whole are not generic: they depend on particular representation of indices and vector updates. Therefore, they are defined as a signature

```
module type VEC = sig
```

```
  type idx
  type unt
  val iter : (idx, unt) vec → unt
end
```

For now, we need only one operation, iter, which is a version of the general reduce for a particular result type unt (which is meant to be a monoid: a set of values with an associative binary operation with a neutral element).

One implementation, for vectors representing ordinary arrays, is:

```
module VecSta = struct
  type idx = int
  type unt = unit

  let iter (Vec (n,body)) =
    for i= 0 to n−1 do
      body i
    done
end
```

4.4 Vector arithmetic DSL

We have all the pieces for the DSL of vector-vector (BLAS 1-like) operations and can reach our goal: clearly written and efficient element-wise vector multiplication.

The two operations below will make the task easier: the vector assignment and the abbreviation for element-wise multiplication[3]:

```
module BLAS1(R: RING)(A: VEC) = struct
  open R
  open A

  let ( := ) vout vin = iter (vec_assign vout vin)
  let ( ∗. ) v1 v2 = zip_with mul v1 v2
end
```

They are parameterized by a RING implementation (which defines multiplication) and a VEC implementation (which tells how to carry out vec_assign on all elements of the output vector).

The desired operation to multiply two vectors v1 and v2 element-wise and put the result in vout takes the form

[3]Recall, (:=) is the function name for the infix := operation. The BLAS1 module hence defines its own version, different from the 'standard' Pervasives.(:=)

```
module VMULT(R: RING)(A: VEC) = struct
  module L = BLAS1(R)(A)
  open L
  let vmult vout v1 v2 =  vout := v1 *. v2
end
```

Ignoring the boilerplate (bringing the just defined convenient operations into scope) the code is clear and Matlab-like. It is obviously correct. We can even try it right away, to multiply up OCaml arrays of complex numbers:

```
let vmult :
    float_complex array → float_complex array → float_complex array → unit =
  fun vout v1 v2 →
    let n = Array.length vout in
    let vout = OVec (n, fun i v → vout.(i) ← v) in
    let v1  = Vec (n,fun i → v1.(i)) in
    let v2  = Vec (n,fun i → v2.(i)) in

    let module V = VMULT(FloatComplex)(VecSta) in
    V.vmult vout v1 v2
```

The code instantiates the generic VMULT with the implementation of float complex operations and maps OCaml arrays to abstract vectors (as was before in vec_of_array). We can apply vmult to three suitable arrays and see the result.

Instantiating VMULT differently or changing the abstract vector mappings gives a version of vmult for a different representation of complex numbers or complex arrays. However, we pay for this generality with multiple levels of indirections: functors, first-class functions, etc. The overhead of abstraction can be eliminated through code generation, shown next.

4.5 Compiling vector DSL

We now interpret the very same VMULT vector multiplication code as a code generator – to be more precise, a family of generators for different memory layouts of complex vectors. The net effect is a compiler for our DSL.

We already have all the parts of the compiler except one: iteration over future-stage arrays. Here we add it:

```
module VecDyn = struct
```

```
  type idx =  int code
  type unt =  unit code

  let iter (Vec (n,body)) =
    .<for i= 0 to (.~n)-1 do
      .~(body .<i>.)
    done>.
end
```

An (int,float) vec is clearly present-stage: given an int index, now, we obtain the corresponding, also present-stage float value. Conversely, (int code, float code) vec is entirely in the future. Even its length, which is int code, is not known now.

Exercise 21. What does (int,float code) vec represent?

Exercise 22. What is the type of the future-stage output vector?

The code to iter-ate over a future vector writes itself once we fix the type of the index, as int code, and of unt, as unit code. We copy the iter code from VecSta and get the type checker to tell where to put the staging annotations.

The same previously written vector-multiplications VMULT can be turned into a generator once instantiated with the code-generating implementations of RING and VEC:

```
let module V =  VMULT(FloatCodeComplex)(VecDyn) in
 V.vmult
⇝ - : (VecDyn.idx, FloatCodeComplex.t, VecDyn.unt) ovec →
    (VecDyn.idx, FloatCodeComplex.t) vec →
    (VecDyn.idx, FloatCodeComplex.t) vec → VecDyn.unt
=  <fun>
```

This code template is turned into code similarly to the way the spower template was converted to the spowern code in §2.2. However, we now have a choice of complex vector representations. Suppose we want the vmult code for the array-of-structure case (that is, float complex arrays). The code generator should have the following signature and the general shape (analogous to spowern from §2.2):

```
let vmult_ca :
  (float_complex array → float_complex array → float_complex array → unit) code
  =  .<fun aout a1 a2 →
    let n =  Array.length aout in
 .~(let vout =  OVec (.<n>., fun i v → . . .  ) in
```

```
    let v1  =  Vec (.<n>.,fun i → ... ) in
    let v2  =  Vec (.<n>.,fun i → ... ) in

    let module V =  VMULT(FloatCodeComplex)(VecDyn) in
    V.vmult vout v1 v2)
  >.
```

We have to define how to represent the future-stage float_complex array as (VecDyn.idx, FloatCodeComplex.t) vec and (VecDyn.idx, FloatCodeComplex.t, VecDyn.unt) ovec needed by the V.vmult template. Recall, (VecDyn.idx, FloatCodeComplex.t) vec maps an int code index to a FloatCodeComplex.t (that is, float code cmplx) value at that index. The first version

```
let vmult_ca :
  (float_complex array → float_complex array → float_complex array → unit) code
  =  .<fun aout a1 a2 →
    let n =  Array.length aout in
  .~(let vout =  OVec (.<n>., fun i v → .<aout.(.~i) ← .~v>. ) in
    let v1  =  Vec (.<n>.,fun i → .<a1.(.~i)>. ) in
    let v2  =  Vec (.<n>.,fun i → .<a2.(.~i)>. ) in

    let module V =  VMULT(FloatCodeComplex)(VecDyn) in
    V.vmult vout v1 v2)
  >.
⇝ Characters 397−401:
    V.vmult vout v1 v2)

Error: This expression has type
         (VecDyn.idx, float_complex code, unit code) ovec
       but an expression was expected of type
         (VecDyn.idx, FloatCodeComplex.t, VecDyn.unt) ovec
       Type float_complex code is not compatible with type
         FloatCodeComplex.t =  RingFloatCode.t Cmplx.cmplx
```

leads to a type mismatch error: float_complex code was found but float code cmplx was needed. Fortunately, we have already defined functions of_complex_code and of_code_complex that convert between these types. Inserting them

```
let vmult_ca :
  (float_complex array → float_complex array → float_complex array → unit) code
  =  .<fun aout a1 a2 →
    let n =  Array.length aout in
  .~(let vout =  OVec (.<n>., fun i v → .<aout.(.~i) ← .~(of_code_complex v)>. ) in
    let v1  =  Vec (.<n>.,fun i → of_complex_code .<a1.(.~i)>. ) in
    let v2  =  Vec (.<n>.,fun i → of_complex_code .<a2.(.~i)>. ) in
```

```
    let module V = VMULT(FloatCodeComplex)(VecDyn) in
    V.vmult vout v1 v2)
  >.
⇝ val vmult_ca :
  (float_complex array → float_complex array → float_complex array → unit)
  code = .<
  fun aout_14 →
    fun a1_15 →
      fun a2_16 →
        let n_17 = Array.length aout_14 in
        for i_18 = 0 to n_17 − 1 do
          aout_14.(i_18) ←
            {im =
               (a1_15.(i_18)).re ∗. (a2_16.(i_18)).im +.
                 (a1_15.(i_18)).im ∗. (a2_16.(i_18)).re;
             re =
               (a1_15.(i_18)).re ∗. (a2_16.(i_18)).re −.
                 (a1_15.(i_18)).im ∗. (a2_16.(i_18)).im
            }
        done>.
```

gives the desired generator. The code looks optimal, with no traces of functors, first-class functions to access array elements or other indirections.

Exercise 23. There is one more optimization one would have liked in the code: bind a1_15.(i_18) to a local variable. How do you think we could insert such a let-statement?

The point of the vector abstraction is to handle different layouts. Let's build the vmult code for the structure-of-array case:

```
let vmult_ac :
    (float array cmplx → float array cmplx → float array cmplx → unit)
    code =
  .<fun aout a1 a2 →
    let n = Array.length aout in
  .~(let vout = OVec (.<n>., fun i v → ... ) in
    let v1  = Vec (.<n>.,fun i → ... ) in
    let v2  = Vec (.<n>.,fun i → ... ) in

    let module V = VMULT(FloatCodeComplex)(VecDyn) in
    V.vmult vout v1 v2)
  >.
```

We now need to represent the future-stage float array cmplx as a (VecDyn.idx, FloatCodeComplex.t) vec vector. We could use the same

old approach: submit the unfinished code to the type checker and let it point out what needs to be done. We quickly obtain:

```
let vmult_ac :
   (float array cmplx → float array cmplx → float array cmplx → unit)
   code =
 .<fun aout a1 a2 →
   let n = Array.length aout.re in
   let a1re = a1.re and a1im = a1.im and a2re = a2.re and a2im = a2.im in
 .~(let vout = OVec (.<n>., fun i v →
                        .<aout.re.(~i) ← .~(v.re); aout.im.(~i) ← .~(v.im)>.) in
    let v1 = Vec (.<n>., fun i →
                        {re= .<a1re.(~i)>.;im= .<a1im.(~i)>.}) in
    let v2 = Vec (.<n>., fun i →
                        {re= .<a2re.(~i)>.;im= .<a2im.(~i)>.}) in

    let module V = VMULT(FloatCodeComplex)(VecDyn) in
    V.vmult vout v1 v2)
 >.
⇝ val vmult_ac :
 (float array cmplx → float array cmplx → float array cmplx → unit)
 code = .<
 fun aout_19 →
   fun a1_20 →
     fun a2_21 →
       let n_22 = Array.length aout_19.re in
       let a1re_23 = a1_20.re and a1im_24 = a1_20.im
       and a2re_25 = a2_21.re and a2im_26 = a2_21.im in
       for i_27 = 0 to n_22 − 1 do
         aout_19.re.(i_27) ←
           a1re_23.(i_27) ∗. a2re_25.(i_27) −.
             a1im_24.(i_27) ∗. a2im_26.(i_27);
         aout_19.im.(i_27) ←
           a1re_23.(i_27) ∗. a2im_26.(i_27) +.
             a1im_24.(i_27) ∗. a2re_25.(i_27)
       done>.
```

Again the code looks optimal, with no trace of abstractions and indirections.

An HPC expert will likely be disappointed though: we have produced OCaml code rather than C or FORTRAN, which are dominant in high-performance. The problem lies not just in extensive optimizations performed by leading C and especially FORTRAN compilers: OCaml libraries are not easy to link with C or FORTRAN code. The ideal is however closer than one may think. The code we have just generated is in OCaml – but in a simple, imperative subset of OCaml, with no higher-order functions, recursion, let alone more advanced facilities. In

short, we have generated FORTRAN-like code. Automatically translating such simplified OCaml code to C or FORTRAN is tantalizing. Such translation – called *offshoring* – has indeed been done on several occasions (Eckhardt et al., 2007; Takashima et al., 2015). The current MetaOCaml already offers some support for offshoring, and may even include such facility in the distribution[4].

4.6 Recap and further challenges

We have begun approaching code generation systematically: get to understand the problem domain (linear algebra, in our case), define abstractions such as RING and VEC appropriate for the domain, write the desired algorithm in terms of these abstractions – and then design interpretations, as ordinary OCaml code (for testing) and as code generators. When designing the domain abstractions, we should think only of what makes the most sense to domain expert. We should not worry about the cost of abstractions in terms of creating closures and functors and indirections – the techniques we have learned will ensure the abstraction costs are born by the generator and disappear in the generated code. The early origin of this approach, defining rings (and fields and arrays of different layouts) can be seen in Carette and Kiselyov (2011), which tackles a quite more involved problem, of generating Gaussian Elimination code, specialized to matrix representation, number domain, inplace operation, and many more aspects.

Vector arithmetic also offers many further exercises and challenges:

- Try other choices for the layout of complex vectors (e.g., C-like Bigarray).
- Try the polar representation of complex numbers.

[4]By its very design, offshoring handles only a *subset* of OCaml. It is difficult to describe that subset in the extant OCaml type system. Therefore, offshoring generated code may, in general, end up in a run-time error. However, if code generation is encapsulated in DSL interpreters, as we have been doing in this section, the DSL developer is in a position to ensure that the generated code in offshorable. This is certainly a burden on the DSL implementor. The end users however can rest assured that every DSL expression is convertible to efficient low-level code.

- Generate code for other complex vector operations, for example, computing $ax_i + y_i$ where a is a scalar. (The common BLAS name for the latter operation is caxpy, 'p' stands for plus.)

All these extensions should be added modularly, reusing the already written code and the already implemented DSL.

5

Linear Algebra DSL: Matrix-Vector Operations and Modular Optimizations

The promise of program generation is producing efficient code from high-level, obviously correct specifications by making it possible and convenient for domain experts to write and apply "optimization passes" at a high-level of abstraction.

On the example of the first Shonan Challenge by high-performance computing to staging (Aktemur et al., 2013) we demonstrate that the promise is fulfilled. We write matrix-vector multiplication in the familiar to domain experts Matlab-like notation – and obtain in the end assuredly well-typed and optimal code specialized to the given mostly sparse matrix. We thus extend the systematic approach to program specialization introduced in the previous section, tackling not just specializations but *modular* optimizations.

MetaOCaml, to maintain static assurances, does not let us inspect, let alone rearrange, the generated code. Nevertheless, optimizations are possible; moreover, they are modular and assuredly sane. They are performed at the level much higher than search-and-replace in the abstract syntax tree or a source code string. The optimizations are developed in small separate modules and mixed in one-by-one, checking their effect after each small step – and are applied to the original code as it

was, without re-writing or even annotating it. They are typed and type preserving, thus ensuring the optimized program remains well-typed. Although the type-preservation per se does not guarantee full correctness, it guarantees sanity. When code is being optimized by advanced domain experts rather than professional compiler writers, it is wise to keep the users from shooting themselves in the foot.

The end result is a small domain-specific language embedded in MetaOCaml that lets us write high-level Basic Linear Algebra expressions and apply several passes of domain-specific optimizations such as selective loop unrolling, sparse matrix representations, and algebraic simplifications.

5.1 Shonan challenge 1

The challenge, which comes from Hidden-Markov Model (HMM) computations in bioinformatics, is to obtain the efficient code for matrix-vector multiplication

$$v'_i = \sum_{k=0}^{n-1} a_{ik} v_k$$

where the matrix a is statically known and mostly sparse, such as:

$$\begin{bmatrix} 0.5 & 0. & 0. & 0.5 & 0. \\ 0. & 0. & 1. & 0. & 0. \\ 0. & 0. & 0.2 & 0.3 & 0.5 \end{bmatrix}$$

In HMM, its elements are transition probabilities. (Strictly speaking, the Shonan Challenge (Aktemur et al., 2013) simplified the problem to the binary case: all elements of a_{ij} are either zero or one, i.e., it is an adjacency matrix.) Some, but not all, rows of the matrix are sparse. In bioinformatics HMM, the number of rows and columns could be as high as 30 000.

The challenge is to:

- build a DSL to express the problem as simply and clearly as $\mathsf{vout} := \mathsf{a} * \mathsf{v};$
- generate the efficient code specialized to the statically known matrix a. The code should be algebraically simplified, that is, contain

no expressions of the form 0 ∗ e, 1 ∗ e, 0 + e, etc. The loop over the matrix rows must be fully unrolled; the inner loop, computing the inner product of v with a matrix row, should be unrolled only if the row is sparse enough (has fewer non-zero elements than a fixed threshold). For the sample matrix above the desired code, assuming the sparsity threshold 3, should look like:

```
let mvmult_ideal : float array → float array → unit =
  fun vout v →
   vout.(0) ← 0.5 ∗. v.(0) + 0.5 ∗. v.(3);
   vout.(1) ← v.(2);
   let sum = ref 0.0 in
     for j= 0 to n−1 do sum ← sum +. a.(2).(j) ∗. v.(j) done;
   vout.(2) ← sum;
  ...
```

The main challenge is to add the optimizations – the outer loop unrolling, selective inner loop unrollings, algebraic simplifications – modularly. The modularity is what distinguishes the solution below from those described in (Aktemur et al., 2013).

5.2 BLAS 2 DSL

Our goal is to obtain the optimal code as a specialization – refinement – of a high-level, obviously correct specification. The first step is to choose the notation for the desired operation to multiply a matrix a by a vector v and store the result in vout. We take the familiar, Matlab-like notation, promised in §1.1:

```
let mvmult vout a v = vout := a ∗ v
```

and specify what it means by literally following the textbook definition: the i-th element of the resulting vector is the dot-product of v with the i-th row of a. Let us write it down in the formal language that can be mechanically (type-) checked, executed and refined.

Recall, §4.2 defined a DSL RING to talk about things may be added and multiplied: the numeric domain of our computations, with operations add, sub, mul and the constants zero and one. We will use the language (code file `ring.ml`) here as it is. We have also introduced, in §4.3, an abstraction for input and output vectors, repeated below for reference:

```
type (ι,α) vec     =  Vec  of ι * (ι → α)
type (ι,α,ω) ovec  =  OVec of ι * (ι → α → ω)
```

where α is the type of vector elements, ι is the index type and ω is a representation of the updated vector. The abstract vector is a mapping from a finite range of indices to values (for an output vector: from an index to an update function). The mapping is paired with the index upper bound (which specifies the range in question: the lower bound is implicit). We have defined the operations vec_map, zip_with, (:=), ($*$.): see §4 and the accompanying code `vector.ml`.

The textbook specification of matrix-vector multiplication regarded the matrix as a vector of rows, which are themselves vectors. The matrix type is hence (ι,(ι,a) vec) vec. We write the specification as:

```
let ( * ) a v =  vec_map (dot v) a
```

where the dot product is yet to be formally defined. The textbook again tells us that to find the dot product of two vectors we multiply them element-wise and sum up, i.e., reduce, the result. The element-wise multiplication was the subject of §4; what is left is reduction.

In fact, §4.3 already introduced a particular form of reduction, called iter:

```
module type VEC =  sig
  type idx
  type unt
  val iter : (idx, unt) vec → unt
end
```

It is defined as a module because its implementation depends on how the vector update unt and the index type idx are actually realized.

We now need a general version of reduction (also called 'left fold'). It makes sense to add it to VEC. We do not want however to barge in and change the already existing code because it may break everything that depends on it; we do not want to keep making copies either. The point of the systematic approach is to get away from the unmaintainable mess of many, slightly different versions of basically the same code. Changes should be introduced modularly, incrementally, and backward-compatibly. Fortunately, the OCaml module system is designed for this. In our case, we define a new module signature, VECR, for vectors not only with iter but also with the general reduction:

```
module type VECR = sig
  include VEC
  type t
  val reduce : (t → t → t) → t → (idx, t) vec → t
end
```

The directive **include** VEC inserts, so to speak, the contents of the earlier defined VEC, to which we add the operation to reduce the elements of a (idx,t) vec to the single value t. The first two arguments of reduce are the add-like function to combine two values of type t, and the zero-like value to return upon the reduction of the empty vector. We implicitly assume that the combining function is associative and zero is its unit element – i.e., that the type t with operations add and zero is a monoid.

Everything is set to write the textbook matrix-vector multiplication formally, in the way that can be later refined and implemented[1]:

```
module BLAS2(R: RING)(A: VECR with type t = R.t) = struct
  open R
  open A
  include BLAS1(R)(A)
  let dot v1 v2 = reduce add zero (v1 *. v2)
  let ( * ) a v = vec_map (dot v) a      (* matrix-vector mul *)
end
```

Matrix-vector operations belong to BLAS Level 2 (BLAS), hence the name BLAS2 for the module implementing the specification. It is written by analogy with BLAS1 in §4.4 and is likewise parameterized by the numerical domain RING and by the implementation that provides iter – and now reduce. There is a new subtlety: the type of values to reduce (sum up) (which is t in VECR) and the type of values to multiply up (the type t in R) must be the same. Thus VECR and RING implementations cannot be completely arbitrary: they must agree on the type t – which is expressed as a *sharing constraint* VECR **with type** t = R.t. We make BLAS2 as an extension of BLAS1 by including BLAS1 for the same RING and vector. An attentive reader must have noticed that A in BLAS2 is an implementation of VECR (vectors with the full reduction operation) whereas the second argument of BLAS1 functor requires VEC. By including VEC within the signature VECR we made sure that

[1]The complete code for the rest of this section is in `mvmult_full.ml`

everything defined by VEC is also defined within VECR: in other words, any implementation of VECR is also an implementation of VEC.

Exercise 24. Is any implementation of BLAS2 also an implementation of BLAS1?

To wrap it up, we formally define an implementation of mvmult in terms of RING and VECR implementations:

```
module MVMULT(R: RING)(A: VECR with type t = R.t) = struct
  module L = BLAS2(R)(A)
  open L
  let mvmult vout a v =
    vout := a * v
end
```

The module MVMULT opens BLAS2 but does not include it[2]. That means MVMULT is not an extension of BLAS2. However, any BLAS2 operation is available within MVMULT, in particular, (:=) and (*) needed for our purposes.

That was the very general, textbook code, which we write once and for all. We will never have to copy or modify it. From now on, we will be instantiating MVMULT with various implementations of RING and VECR, obtaining more and more optimized matrix-vector multiplication code.

5.3 Implementing and generating matrix-vector multiplication

The just defined MVMULT gives an implementation of matrix-vector multiplication, provided we have implementations of RING and reducible vectors. Let us see.

Let's take the RingFloat, and the VecSta implementation of VEC from §4, and extend VecSta to realize the VECR interface:

```
module VecRSta(T:sig type t end) = struct
  include T
  include VecSta
```

[2]Although we can write include BLAS2(R)(A), for subtle technical reasons we cannot (yet) write open BLAS2(R)(A) and so have to use a more verbose notation, first binding BLAS2(R)(A) to the name L.

```
  let reduce plus zero (Vec (n,body)) =
    let sum = ref zero in
      for i = 0 to n−1 do
        sum := plus !sum (body i)
      done;
      !sum
end
```

VECR adds to VEC the type t of values to reduce and the function reduce. These are the two components to add to VecSta to make it an implementation of VECR. For generality, we do not pick a realization for t; rather, we leave it as a parameter (enclosed in the trivial signature T).

We immediately obtain matrix-vector multiplication with vectors and matrices represented as ordinary OCaml arrays (mapped to abstract vectors as shown in §4.4):

```
let mvmult_p : float array → float array array → float array → unit =
  fun vout a v →
    let n = Array.length vout and m = Array.length v in
    let vout = OVec (n,fun i v → vout.(i) ← v) in
    let v   = Vec (m,fun j → v.(j)) in
    let a   = Vec (n,fun i → Vec (m, fun j → a.(i).(j))) in

    let module MV = MVMULT(RingFloat)(VecRSta(RingFloat))
    in MV.mvmult vout a v
```

Recall VecRSta is parameterized by **type** t of values to reduce: it requires a module with the field **type** t. RingFloat is such module (which, not surprisingly, realizes type t in such a way that it matches **type** t in, well, RingFloat itself – as required by the sharing constraint)[3].

Let's test the just obtained mvmult_p: define a sample matrix

```
let a = [|
    [| 0.5; 0.; 0.; 0.5; 0. |];
    [| 0.; 0.; 1.; 0.; 0. |];
    [| 0.; 1.; 0.; 0.; 0. |];
    [| 0.; 0.; 0.2; 0.3; 0.5 |];
    [| 0.; 0.; 0.3; 0.; 0.7 |]
  |]
```

and a sample vector, compute the output

[3] One perhaps gets the intimation that naming the implementation type uniformly as t in all modules is not just a failure of imagination.

```
let v1 = [| 1.; 2.; 3.; 4.; 5. |]
let v1out =
  let vout = Array.make 5 0. in
  mvmult vout a v1; vout
```

and check that it matches the expected value

```
let [| 2.5; 3.; 2.; 4.3; 4.4 |] = v1out
```

(It is convenient to write a unit test as a let-statement with a constant left-hand-side, which is a pattern-matching in disguise. If v1out does not match the constant array, an exception is raised).

Exercise 25. How difficult would it be to implement the multiplication of a complex-valued matrix to a complex-valued vector? Do we have all the pieces already?

Although mvmult_p is general and conveniently obtained, it is hardly efficient. For example, the input arrays are addressed indirectly, through the abstract vector interface, with a function call per element access. That is where code generation comes in. From the very same MVMULT specification we shall obtain code free from any abstraction overhead. Almost all pieces are at hand: the RingFloatCode implementation of RING and the VecDyn of VEC, from §4; the latter is extended to implement VECR as we did for VecSta earlier:

```
module VecRDyn(T:sig type t end) = struct
  include T
  include VecDyn

  let reduce : (α code → α code → α code) → α code →
               (int code, α code) vec → α code =
    fun plus zero (Vec (n,body)) →
    .<let sum = ref .~zero in
      for i = 0 to (.~n)−1 do
        sum := .~(plus .<!sum>. (body .<i>.))
      done;
      !sum>.
end
```

VecRDyn.reduce is the staged version of VecRSta.reduce, derived in the type-driven approach that we learned and practiced in §2 and §3.

Exercise 26. Implement reduce in VecRSta as a tail-recursive function, and stage it to obtain VecRDyn.reduce

When instantiated with RingFloatCode, VecRDyn and the mapping from future-stage arrays to abstract vectors (the same as in §4.5), the MVMULT specification now turns into a code generator:

```
let mvmult_c : (float array → float array array → float array → unit) code =
 .<fun vout a v →
   let n =  Array.length vout and m =  Array.length v in
   .~(let vout =  OVec (.<n>.,fun i v → .<vout.(.~i) ← .~v>.) in
      let v    =  Vec  (.<m>.,fun j → .<v.(.~j)>.) in
      let a    =  Vec  (.<n>.,fun i →
                          Vec (.<m>., fun j → .<a.(.~i).(.~j)>.)) in

      let module MV =  MVMULT(RingFloatCode)(VecRDyn(RingFloatCode))
      in MV.mvmult vout a v)
 >.
```

Whereas mvmult_p is a present-stage function, mvmult_c is code, namely:

```
val mvmult_c : (float array → float array array → float array → unit) code =
 .<fun vout_4 a_5 v_6  →
      let n_7 =  Array.length vout_4 and m_8 =  Array.length v_6 in
      for i_9 =  0 to n_7 − 1 do
        vout_4.(i_9) ←
          (let sum_10 =  Pervasives.ref 0.  in
           for i_11 =  0 to m_8 − 1 do
             sum_10 :=  (! sum_10) +. v_6.(i_11) ∗. a_5.(i_9).(i_11)
           done;
           ! sum_10)
      done>.
```

The arrays vout_4, a_5 and v_6 are now accessed as they are, with no indirections: all overhead of rings and abstract vectors is gone.

The code can be saved to a file for later use – or tested now:

```
let [| 2.5; 3.; 2.; 4.3; 4.4 |] =
  let vout =  Array.make 5 0.0 in
  (run mvmult_c) vout a v1; vout
```

The challenge is to specialize the code to the known sparse matrix (like the matrix a earlier) and optimize accordingly. That is what we will be doing next, in small steps.

5.4 Specializing to the known dimensions

The Challenge states that the matrix to multiply with is known (before the program runs, that is). Its dimensions, and hence the sizes of the

input and the output vectors, are all known. As the first step, we take advantage of this knowledge – as we did in §3.1 when specializing filters to the known order and hence the known 'spine'.

Previously we instantiated MVMULT with RingFloat and the int-indexed vector with float elements – obtaining the matrix-vector multiplication function mvmult_p to use now. We also instantiated MVMULT with the RingFloatCode that merely promised to add or multiply integers, and with the VECR that only promised to index a vector. The result was mvmult_c that can only be used later, after compiling it – although we can see its code now. It had no optimizations. A mixed choice is a 'hybrid' vector with int indices but float code elements: we can obtain its elements now but can only use them for later computations. The length of a hybrid vector, an int, is also known now, that is, when the generator runs. Thus we obtain the code for multiplying a matrix by a vector of statically known dimensions.

The hybrid vectors are represented by the following structure, which is quite like VecRSta since in both cases the indices and dimensions are usable at generation time. To implement the VECR interface, VecRStaDim below defines the realizations for the types unt, idx and t (the latter by including T). Clearly, the type t must be a code type. VecRStaDim also has to implement reduce and iter. For the former, we reuse VecRSta's, which reduces either float or float code elements depending on the combining function it receives as the argument.

```
module VecRStaDim(T: sig type t end) = struct
  include T
  module M = VecRSta(T)
  type idx = int
  type unt = unit code

  let reduce = M.reduce

  let seq e1 e2 = .<.~e1; .~e2>.
  let iter arr = reduce seq .<()>. arr
end
```

The operation iter is defined as a particular case of reduce, as we have intimated earlier.

By replacing VecRDyn with VecRStaDim in mvmult_c (and accounting for the fact that matrix dimensions n and m are now the present-

stage integers) we obtain (with changes highlighted)

```
let mvmult_nc : int → int →
  (float array → float array array → float array → unit) code =
 fun n m → .<fun vout a v →
   assert (n = Array.length vout && m = Array.length v);
   .~(let vout = OVec (n,fun i v → .<vout.(i) ← .~v>.) in
     let v  = Vec (m,fun j → .<v.(j)>.) in
     let a  = Vec (n,fun i → Vec (m, fun j → .<a.(i).(j)>.)) in
     let module MV = MVMULT(RingFloatCode)(VecRStaDim(RingFloatCode))
     in MV.mvmult vout a v)
  >.
```

Although essentially like mvmult_c, the new generator is a function that takes the matrix dimensions as the arguments and returns the specialized code. For example,

```
let mvmult_5c = mvmult_nc 5 5
```

produces:

```
val mvmult_5c :
  (float array → float array array → float array → unit) code = .<
  fun vout_38 a_39 v_40 →
      assert ((5 = Array.length vout_38) && (5 = Array.length v_40));
      vout_38.(0) ←
          0. +. a_39.(0).(0) *. v_40.(0) +.
              a_39.(0).(1) *. v_40.(1) +.
              a_39.(0).(2) *. v_40.(2) +.
              a_39.(0).(3) *. v_40.(3) +.
              a_39.(0).(4) *. v_40.(4);
      vout_38.(1) ←
          0. +. a_39.(1).(0) *. v_40.(0) +.
              a_39.(1).(1) *. v_40.(1) +.
              a_39.(1).(2) *. v_40.(2) +.
              a_39.(1).(3) *. v_40.(3) +.
              a_39.(1).(4) *. v_40.(4);
 ...>.
```

Exercise 27. Explain the purpose of the generated **assert** statements

All loops are fully unrolled. This is what it means to take advantage of the statically known dimensions, isn't it?

5.5 Specializing to the known matrix: Partially-known values

The just generated the code took the full advantage of the statically known dimensions: unrolling all loops and statically indexing all vectors. In the Challenge, the matrix itself is also statically known. To

specialize mvmult to the known matrix we breed another mongrel vector, whose elements and indices are the hybrid of the present and the future stages.

All values of the type float are usable at generation time; all values of the type float code can only be used later. Among *partially known* values, some are usable now, and some later:

```
type α pv = Sta of α | Dyn of α code
```

A partially known value *may* be fully known at the generation time. In either case, it is known at the future stage, as witnessed by the following dyn function that 'forgets' what we may know and returns the opaque, code value.

Actually, we have promised in §3.2.1 to be explicit about transporting – or 'lifting', as it is usually called – of values from the present to the future. We have mentioned the LIFT signature (repeated below)

```
module type LIFT = sig
  type t
  val lift: t → t code
end
```

and several of its implementations such as Lift_float for lifting values of specific types – all collected in `lift.ml`. Strictly speaking, the dyn function is only possible (and makes sense) for liftable types, i.e., for which there is an implementation of LIFT:

```
module Dyn(L:LIFT) = struct
  let dyn : L.t pv → L.t code = function
  | Sta x → L.lift x
  | Dyn x → x
end
```

Exercise 28. Even more strictly speaking, partially known values can be defined differently, as a product (record) rather than a sum:

```
type α pv = {sta: α option; dyn: α code}
```

That is, we always know the opaque, future-stage form of the value – and we may also know the value fully at the present stage. The function dyn then is always defined (as the dyn projection from the record), whether the type α is liftable or not. Can you surmise why in our challenge this generality is beside the point and we would still need lift anyway?

Since we will often be dealing with integer (indices) and floats (which are all liftable), we define the following dyn instances for them:

```
let dyni : int pv   → int code   =  let module M= Dyn(Lift_int)   in M.dyn
let dynf : float pv → float code =  let module M= Dyn(Lift_float) in M.dyn
```

Just like float and float code can be put in a ring, so can their hybrid. We show it below in the general case, by combining two rings, one (referred to as STA) for t and the other for t code, where t is an arbitrary liftable type:

```
module RingPV(STA:RING)(DYN:RING with type t =  STA.t code)
            (L:LIFT with type t =  STA.t) =
struct
  type t =  STA.t pv
  include Dyn(L)

  let zero =  Sta STA.zero
  let one  =  Sta STA.one
  let add x y =
    match (x,y) with
    | (Sta x, Sta y) → Sta STA.(add x y)
    | (x, y) → Dyn DYN.(add (dyn x) (dyn y))
  let sub x y =  ...
  let mul x y =  ...
end
```

The ring indeed looks like a cross-breed of STA and DYN. For example, when adding two partially known values, if both summands are known now, we sum them now, using add of the STA ring. (The notation STA.(add x y) is the abbreviation for **let open** STA **in** add x y, which in our case is equivalent to STA.add x y.) Otherwise, we build the code to add them later with DYN.add; since it takes the arguments of code types, we have to convert STA.t pv to STA.t code, using dyn provided by the Dyn(L) module. Clearly, zero and one are known now. Partially known values are the staple of code generation and partial evaluation in general, and should be a library feature. We implemented them here for completeness.

Exercise 29. Fill in the missing pieces in the code

For ease of reference we define the following abbreviation, an instance of RingPV for floats:

```
module RingFloatPCode =  RingPV(RingFloat)(RingFloatCode)(Lift_float)
```

Our mongrel vectors, VecRStaDyn below, are the cross-breed of VecRStaDim and VecRDyn. They have partially-known values both as indices (length) and as elements. If the length is statically known, we dispatch to the VecStaDim implementation, written for this case. Otherwise, we treat the vector as statically unknown and use the general VecDyn. From now on, our development would be to mix-and-slightly-extend the previously written code.

```
module VecRStaDyn(T: LIFT) = struct
  type t = T.t pv
  include Dyn(T)
  type idx = int pv
  type unt = unit code

  module VSta = VecRStaDim(T)
  module VDyn = VecRDyn(T)

  let reduce plus zero = function
    | Vec (Sta n,v) →
        VSta.reduce plus zero (Vec (n, fun i → v (Sta i)))
    | Vec (Dyn n,v) →
      Dyn (VDyn.reduce (fun x y → dyn (plus (Dyn x) (Dyn y))) (dyn zero)
            (Vec (n,fun i → dyn (v (Dyn i)))))

  let iter = ...
end
```

The new generator instantiates the same old MVMULT with mongrel vectors. Since the matrix is regarded as statically known, indexing it with now known indices gives the now usable value. The input vector v, albeit of the statically known length, comprises elements unknown at the generator time (hence tagged as Dyn):

```
let mvmult_ac : float array array → (float array → float array → unit) code =
 fun a →
   let n = Array.length a in
   let m = Array.length a.(0) in
   let a = Vec (Sta n, fun i → Vec (Sta m,
    (fun j →
      match (i,j) with
      | (Sta i, Sta j) → Sta a.(i).(j)
      | (Sta i, Dyn j) → Dyn .<a.(i).(.~j)>.
      | (i, j) → Dyn .<a.(.~(dyni i)).(.~(dyni j))>.))) in
   .<fun vout v →
    assert (n = Array.length vout && m = Array.length v);
   .~(let vout = OVec (Sta n, fun i v → .<vout.(.~(dyni i)) ← .~(dynf v)>.) in
      let v   = Vec (Sta m, fun j → Dyn .<v.(.~(dyni j))>.) in
      let module MV = MVMULT(RingFloatPCode)(VecRStaDyn(Lift_float))
```

```
    in MV.mvmult vout a v)
  >.
```

Compared to the earlier mvmult_nc, the generator mvmult_ac receives not just the matrix dimensions but the matrix itself to specialize to.

From now on, optimizations will be instantiating MVMULT with more 'elaborated' versions of RingFloatPCode and VecRStaDyn, and with more refined mappings of the matrix a to the abstract vectors. The abstract mapping of arrays v and vout, and the overall shape of the mvmult_ac generator remains the same. To avoid repeating the same code over and over again, we abstract away the variable parts of mvmult_ac: mvmult and the a mapping. Remember, with code generation, abstraction, no matter how advanced, costs nothing.

To conveniently abstract away the 'abstract' matrix, we introduce the record

```
type amat = {n: int; m: int; a: (int pv, (int pv, float pv) vec) vec}
```

which carries the known matrix dimensions along with the abstract a itself.

Parameterized with mvmult and the abstract a, the mvmult_ac generator becomes:

```
let mvmult_abs : _ →
 amat → (float array → float array → unit) code =
 fun mvmult → fun {n;m;a} →
   .<fun vout v →
    assert (n = Array.length vout && m = Array.length v);
   .~(let vout = OVec (Sta n, fun i v → .<vout.(.~(dyni i)) ← .~(dynf v)>.) in
     let v   = Vec (Sta m, fun j → Dyn .<v.(.~(dyni j))>.) in
     mvmult vout a v)
  >.
⇝ val mvmult_abs :
     ((int pv, float pv, unit code) Vector.ovec →
      (int pv, (int pv, float pv) Vector.vec) Vector.vec →
      (int pv, float pv) Vector.vec → unit code) →
     amat → (float array → float array → unit) code = <fun>
```

(The notation {n;m;a} in the argument expression is the shorthand for {n= n;m= m;a= a} that makes the three fields of the amat record accessible through the variables of the same name.)

We merely replaced the 'variable parts' of mvmult_ac with the variables mvmult, n, m and a (highlighted), which become the generator arguments. The type of mvmult is rather tedious to write. Therefore,

we cheated, writing it just as an underscore in the function's signature, asking (Meta)OCaml to infer it. We should still look through the inferred type to confirm it is sane.

One instance of amat is the code from mvmult_ac that we have abstracted away:

```
let amat1 : amat =
  let n = Array.length a and m = Array.length a.(0) in
  {n= n; m= m;
   a = Vec (Sta n, fun i → Vec (Sta m,
    (fun j →
      match (i,j) with
      | (Sta i, Sta j) → Sta a.(i).(j)
      | (Sta i, Dyn j) → Dyn .<a.(i).(.~j)>.
      | (i, j) → Dyn .<a.(.~(dyni i)).(.~(dyni j))>.)))
 }
```

The original mvmult_ac is reproduced by supplying to mvmult_abs the abstracted away pieces: amat1 and the instance of MVMULT:

```
let mvmult_ac1 =
  mvmult_abs
   (let module MV =
        MVMULT(RingFloatPCode)(VecRStaDyn(Lift_float)) in MV.mvmult)
   amat1
```

which produces the following code:

```
val mvmult_ac1 : (float array → float array → unit) code = .<
  fun vout_23 v_24 →
      assert ((5 = Array.length vout_23) && (5 = Array.length v_24));
      vout_23.(0) ←
        0. +. v_24.(0) *. 0.5 +. v_24.(1) *. 0. +. v_24.(2) *. 0.
          +. v_24.(3) *. 0.5 +. v_24.(4) *. 0.;
      vout_23.(1) ←
        0. +. v_24.(0) *. 0. +. v_24.(1) *. 0. +. v_24.(2) *. 1.
          +. v_24.(3) *. 0. +. v_24.(4) *. 0.;
...>.
```

Not only loops are unrolled, the matrix elements are inlined. As the result, we see lots of trivial multiplications and additions. Algebraic simplifications are direly needed.

5.6 Algebraic simplifications

Adding algebraic simplifications as an optimization turns out simple and modular, reusing the earlier code. All we need is to adjust the ring, RingFloatPCode, overriding, or improving its operations:

```
module RingFloatOPCode = struct
  include RingFloatPCode
  let add x y =
    match (x,y) with
    | (Sta 0.,y) → y
    | (x,Sta 0.) → x
    | (x,y)      → RingFloatPCode.add x y
  let sub x y = ...
  let mul x y =
    match (x,y) with
    | (Sta 0.,_) → Sta 0.
    | (_,Sta 0.) → Sta 0.
    | (Sta 1.,y) → y
    | (x,Sta 1.) → x
    | (x,y)      → RingFloatPCode.mul x y
end
```

The improvement adds cases to add, sub and mul. For example, when one of the multiplicands is statically zero, the product is statically zero regardless of the other multiplicand. When one multiplicand is known to be one, the product is the other multiplicand. The original RingFloatPCode.mul deals with the rest of the cases.

Exercise 30. Can the optimizations be formulated in terms of the general RingPV rather than just for floats?

Merely by replacing RingFloatPCode with RingFloatOPCode in the earlier generator (the change is highlighted):

```
let mvmult_opt =
 mvmult_abs
  (let module MV =
       MVMULT(RingFloatOPCode)(VecRStaDyn(Lift_float)) in MV.mvmult)
  amat1
```

gives the following optimized code specialized to the sample matrix. It became short enough to show completely:

```
val mvmult_opt : (float array → float array → unit) code = .<
  fun vout_25 v_26 →
      assert ((5 = Array.length vout_25) && (5 = Array.length v_26));
      vout_25.(0) ← v_26.(0) *. 0.5 +. v_26.(3) *. 0.5;
      vout_25.(1) ← v_26.(2);
      vout_25.(2) ← v_26.(1);
      vout_25.(3) ← v_26.(2) *. 0.2 +. v_26.(3) *. 0.3 +. v_26.(4) *. 0.5;
      vout_25.(4) ← v_26.(2) *. 0.3 +. v_26.(4) *. 0.7
>.
```

We have almost reached the goal. However, the Challenge specified that the inner-product loop is to be unrolled only if the corresponding matrix row is sparse enough. Unrolling should be selective.

5.7 Selective unrolling

We have been unrolling all loops with statically known bounds. Unrolling a large loop however may lead to code explosion. A smarter strategy is first to estimate how big the code may become. In matrix-vector multiplication, the size of the unrolled dot-product loop depends on the number of non-zero elements in the vector to reduce. The smart reduce, required by the Challenge, will estimate that number first, and unroll only if it is under a set threshold.

As with the algebraic simplification, we improve the previously written code reusing as much as possible. Here we smarten-up VecRStaDyn by adding a few checks to its reduce:

```
module VecRStaOptDynFloat = struct
  module R = RingFloatOPCode
  module M = VecRStaDyn(Lift_float)
  include M
  let threshold = 3

  let count_non_zeros n vecf =
    let rec loop acc i =
      if i ≥ n then acc else
      let acc = if vecf (Sta i) = Sta 0. then acc else acc + 1 in
      loop acc (i+1)
    in loop 0 0

  let reduce plus zero = function
    | (Vec (Sta n,vecf)) as vec →
        if count_non_zeros n vecf < threshold then
         M.reduce plus zero vec
        else
         M.reduce plus zero (Vec (Dyn .<n>.,vecf))
    | vec → M.reduce plus zero vec
end
```

If the unrolling is at all possible – the length of the vector is known – we check how many elements of the vector are *statically* known to be zero. (Some of the unknown at present stage elements, which are described by the opaque code values, may also turn out zeros.) The

rest are counted as non-zeros. If the count is above the threshold, we make the length of the vector to be statically unknown by tagging it with Dyn, effectively switching off the unrolling.

Using this improved implementation of VecRStaDyn

```
let mvmult_roll =
  mvmult_abs
   (let module MV = MVMULT(RingFloatOPCode)(VecRStaOptDynFloat) in
       MV.mvmult)
  amat1
```

we solve the challenge, attaining the desired mvmult_ideal from §5.1:

```
val mvmult_roll : (float array → float array → unit) code = .<
  fun vout_19 v_20 →
      assert ((5 = Array.length vout_19) && (5 = Array.length v_20));
      vout_19.(0) ← v_20.(0) ∗. 0.5 +. v_20.(3) ∗. 0.5;
      vout_19.(1) ← v_20.(2);
      vout_19.(2) ← v_20.(1);
      vout_19.(3) ←
        (let sum_21 = Pervasives.ref 0. in
          for i_22 = 0 to 5 − 1 do
            sum_21 := (! sum_21) +. v_20.(i_22) ∗. (∗ CSP a ∗).(3).i_22
          done;
          ! sum_21);
      vout_19.(4) ← v_20.(2) ∗. 0.3 +. v_20.(4) ∗. 0.7>.
>.
```

That CSP a, albeit innocent (because a is top-level) is better avoided. We should strive to make the code self-contained.

5.8 Cross-stage persistence for large data

Our last task is to insert the matrix a itself into the generated code – at least, those of its rows whose dot product was not unrolled. (The code of the unrolled dot-product loops already includes the values of the corresponding elements of a: see the mvmult_roll code above.) The generated code will thus become self-contained, with no references to external (OCaml) data – making it possible to later convert it to C via offshoring (talked about at the end of §4.5).

Converting present-stage matrix rows to the corresponding code is cross-stage persistence, or lifting, discussed in §3.2.1. In this section we will look into cross-stage persistence of large data such as long arrays and matrices. The large size of data should make us think about the

size of the generated code to represent the data, and about the memory needed to run it.

First, however, we should think where *(∗ CSP a ∗)* in the mvmult_roll code comes from. Since all matrix access is encapsulated in amat, let's look at the amat record we have been using so far, repeated below for reference:

```
let amat1 : amat =
  let n = Array.length a and m = Array.length a.(0) in
  {n= n; m= m;
   a = Vec (Sta n, fun i → Vec (Sta m,
    (fun j →
      match (i,j) with
      | (Sta i, Sta j) → Sta a.(i).(j)
      | (Sta i, Dyn j) → Dyn .<a.(i).(.~j)>.
      | (i, j) → Dyn .<a.(.~(dyni i)).(.~(dyni j))>.)))
 }
```

The culprit is .<a.(i).(.~j)>., with the present-stage matrix identifier a appearing within brackets. To avoid the CSP reference, the generated code should refer to the i-th row of a by value, so to speak: the code should have its own copy of the needed a row, and use that.

Let's introduce an operation of the type

```
type copy_row_t = float array → (int code → float code)
```

for the task of 'copying' a matrix row into the generated code – to be more precise, for building code with the contents of a given array. It returns the code template for dereferencing the copied array (that is, obtaining the value at a given index). We made copy_row a bit more general, to encapsulate the representation of the future-stage array, letting us adjust it as needed. Again, we should not be afraid of generality: we are not introducing any overhead in the generated code. Abstraction is really free.

The new matrix access operation, when asked for an element from a statically known matrix row (but statically unknown column), uses copy_row to first insert the row into the generated code, and then builds the code to access the needed column (the changes are highlighted):

```
let amatcopy : copy_row_t → amat = fun copy_row →
  let n = Array.length a and m = Array.length a.(0) in
  {n= n; m= m;
   a = Vec (Sta n, fun i → Vec (Sta m,
```

```
    (fun j →
      match (i,j) with
      | (Sta i, Sta j) → Sta a.(i).(j)
      | (Sta i, Dyn j) → let deref = copy_row a.(i) in Dyn (deref j)
      | (i, j) → failwith "not␣implemented␣yet:␣exercise")))
  }
```

Exercise 31. Implement the missing interface (hint: we need to essentially copy all the rows of the matrix, avoiding copying those that have been copied with copy_row)

The first attempt at copy_row is straightforward: take a float array and convert it to float array code, returning the code template for indexing in the latter array. We have already done a rather similar conversion: dyn_arr_of_spine_arr in §3.3.

```
let copy_row1 : copy_row_t = fun v →
  let vcode =
    .<Array.of_list
      .~(List.fold_right (fun h t → .<.~h :: .~t>.)
              (List.map Lift_float.lift (Array.to_list v))
              .<[]>.)>.
  in fun idx → .<.~vcode.(.~idx)>.
```

The code is self-explanatory: convert float array to float list, map Lift_float.lift to obtain float code list, reduce to float list code and convert it to float array code.

Exercise 32. How to make copy_row generic, not just for floats?

Replacing amat1 with amatcopy in the mvmult_roll generator of the previous section as:

```
let mvmult_let1 =
  mvmult_abs
  (let module MV = MVMULT(RingFloatOPCode)(VecRStaOptDynFloat) in
   MV.mvmult)
   (amatcopy copy_row1)
```

produces the following code:

```
val mvmult_let1 : (float array → float array → unit) code = .<
  fun vout_23 v_24 →
      assert ((5 = Array.length vout_23) && (5 = Array.length v_24));
      vout_23.(0) ← v_24.(0) *. 0.5 +. v_24.(3) *. 0.5;
      vout_23.(1) ← v_24.(2);
      vout_23.(2) ← v_24.(1);
```

```
vout_23.(3) ←
  (let sum_25 = Pervasives.ref 0. in
    for i_26 = 0 to 5 − 1 do
      sum_25 :=
        (! sum_25) +.
          v_24.(i_26) ∗.
            (Array.of_list [0.; 0.; 0.2; 0.3; 0.5]).(i_26)
    done;
    ! sum_25);
vout_23.(4) ← v_24.(2)) ∗. 0.3) +. v_24.(4)) ∗. 0.7>.
```

(the change from mvmult_roll is highlighted). Although it has the correct behavior, something is amiss: the array allocation Array.of_list [0.; 0.; 0.2; 0.3; 0.5] is performed on each iteration of the loop, and the allocated array becomes garbage immediately after one of its elements is retrieved. For long arrays (of real-life HMM problems) such an inordinate waste is a show-stopper. The array in question is constant. Therefore, it should be allocated outside of the loop – even outside the function.

Exercise 33. Have we encountered such a situation before? How should we go about it?

If we wrote code like mvmult_let1 by hand, we would have allocated the array outside the function, let-bound it to a variable and used that variable in the loop. Luckily, the recent MetaOCaml offers exactly this operation to let-bind the result of an expression, with the let-binder placed as outside as possible (but still within the scope of the binders for the free variables of the expression, if any). The operation is called genlet[4]. Here is the new version of copy_row (differing only in genlet):

```
let copy_row_let : copy_row_t = fun v →
  let vcode = Print_code.genlet
    .<Array.of_list
      .~(List.fold_right (fun h t → .<.~h :: .~t>.)
            (List.map Lift_float.lift
             (Array.to_list v))
            .<[]>.)>.
  in fun idx → .<.~vcode.(.~idx)>.
```

the corresponding generator:

[4]genlet is a rather new facility of MetaOCaml and its interface may change. For example, it may be defined in a different library module. Please read the release notes for your version of MetaOCaml.

```
let mvmult_let2 =
  mvmult_abs
  (let module MV = MVMULT(RingFloatOPCode)(VecRStaOptDynFloat) in
   MV.mvmult)
   (amatcopy copy_row_let)
```

and the generated code (the changes are highlighted):

```
val mvmult_let2 : (float array → float array → unit) code = .<
  let lv_31 = Array.of_list [0.; 0.; 0.2; 0.3; 0.5] in
  fun vout_27 v_28 →
      assert ((5 = Array.length vout_27) && (5 = Array.length v_28));
      vout_27.(0) ← v_28.(0) ∗. 0.5 +. v_28.(3) ∗. 0.5;
      vout_27.(1) ← v_28.(2);
      vout_27.(2) ← v_28.(1);
      vout_27.(3) ←
        (let sum_29 = Pervasives.ref 0. in
          for i_30 = 0 to 5 − 1 do
            sum_29 := (! sum_29) +. v_28.(i_30) ∗. lv_31.(i_30)
          done;
          ! sum_29);
      vout_27.(4) ← v_28.(2) ∗. 0.3 +. v_28.(4) ∗. 0.7>.
```

We have attained the ideal.

Since the code is now self-contained and C-like (imperative, with no inner or higher-order functions), it could be rather straightforwardly converted to C, e.g., using offshoring. We do not show this C code (offshoring is still an experimental facility) but it is not so difficult to imagine.

Exercise 34. Although the arrays (matrix rows) to copy_row should be dense enough (otherwise, their dot-product loop would have been unrolled), there may still be many zeros, which take space in the generated code. Change copy_row to use a representation that avoids storing zeros, still allowing relatively cheap access to the elements.

Exercise 35. We have always assumed that the known matrix a is represented as a nested float array array, typical in OCaml. The Challenge says that the matrix is large and mostly sparse. Consider a more economical representation (e.g., adjacency lists). How much of the generator has to be changed to use the compactly-represented a? Would the generated code change in any way?

Exercise 36. The amatcopy code has a big drawback, which luckily has

not been relevant: if the user of a accesses the same row of a twice, it would be copied twice. Find a way to avoid that.

Exercise 37. (Advanced) Detect that a matrix is accessed sequentially and do something smarter to simplify the access.

5.9 Recap

Once again we have encountered the generality-performance trade-off: MVMULT is general, concise and obviously correct, being a restatement of the textbook specification. Its naive realization mvmult_p is inefficient due to indirect array accesses through abstraction layers. The highly specialized and efficient code mvmult_let2, on the other hand, is harder to write by hand and harder to see correct. It is particularly hard to maintain the correctness as we keep adjusting the specialized code to account for changes in a (which often happen in the course of bioinformatics experiments).

In this section we have seen the resolution of the trade-off: the performant code mvmult_let2 is obtained mechanically, from the very same MVMULT. If the matrix a is changed, all we need to do is to re-evaluate mvmult_let2 to obtain the new, just as specialized version.

Kiselyov et al. (2004) was the first to demonstrate how to optimize code when we cannot even examine it. It has not been clear however if optimizations can be programmed modularly, like compiler passes. This also turns out possible, as we have seen in this section. The optimizations were introduced gradually, at a high abstraction level, and they are type-checked, hence ensuring the resulting code shall compile. Optimizations have been programmed in layers, as reusable modules. Since the representation of the generated code is not exposed and its rearrangements are prohibited, we had no choice but to develop optimizations in terms of the appropriate high-level properties of the code. The high level and the compositionality of optimizations lend itself to writing optimization libraries by domain experts.

6

From an Interpreter to a Compiler: DSL for Image Manipulation

A common application of staging is building DSL compilers. First, one implements the DSL as an interpreter; its staging yields a compiler. A computationally intensive image-processing DSL clearly benefits from compilation. Our tutorial problem is a very simple version of the image-processing DSL compiler described in Herrmann and Langhammer (2006).

6.1 Image-processing DSL

Our gray-scale image processing DSL is fairly minimal, conventional, untyped. It is defined by the following module signature, embedded in OCaml in the tagless-final style (the code is in `imgdsl.mli`):

```
module type IMGDSL = sig
  type exp

  val int: int → exp
  val load: string → exp
  val display: exp → exp
  val it: exp
  val ( +% ): exp → exp → exp
  val ( −% ): exp → exp → exp
  val ( ∗% ): exp → exp → exp
  val ( <% ): exp → exp → exp
```

```
  val ( >% ): exp → exp → exp
  val ( =% ): exp → exp → exp
  val if_: exp → exp → exp → exp
  val iterate: exp → exp → exp
end
```

Here, exp is the type of DSL expressions. The language has integer literals, arithmetic and boolean expressions (whose operations are notated as +%, ∗%, >%, etc.) and the conditional expression. There are operations to load an image from a file and to display it. The only unfamiliar iterate takes an image and an expression and evaluates the latter on each pixel of the image, binding it to the current pixel value. The expression result, which must be an integer, is stored in the current pixel. The following DSL expression

```
display(
  iterate (load "takaosan.pgm")
    (if_ (it >% int 128)
        (int 2 ∗% (it −% int 128))
        (int 0)))
```

loads an image, thresholds and enhances its luminance, and displays the result.

6.2 Interpreting DSL

The signature IMGDSL tells what operations the DSL has – but not what they mean. The meaning is given by an an implementation, which we develop now. (The corresponding code is in `img_interp.ml`.)

First we should decide on the representation of DSL expressions in OCaml. DSL has integers, booleans and images, thus:

```
type value = Int of int | Bool of bool | Img of img
```

where img is an abstract type defined in a small library of grayscale image processing `grayimg.mli` and `grayimg.ml`. The library has operations read_pgm to load an img from a file, display it, and access image pixels as a linear array (Bigarray.Array1, to be precise, of OCaml's bigarray library), in a row-wise order.

A DSL expression is hence may be represented as a function

```
type exp = value → value
```

which receives the value associated with it and computes the value of the expression.

We now implement the DSL operations one by one. The operation int turns an OCaml integer into a DSL expression: int x is hence a function that receives the current value of it (which it does not need) and returns x as the DSL value, appropriately tagged:

```
let int x  =  fun _ → Int x
```

Loading an image is interpreted similarly, using the grayimage library to read in the image from the file:

```
let load x =  fun _ → Img (read_pgm x)
```

The operation display e first has to evaluate its argument expression, by passing it the it value. Then we check the result: if an image, it is displayed; otherwise, an error is reported.

```
let display e =  fun itv →
  match e itv with
  | Img x → Grayimg.display x; Int 0
  | _     → failwith "Display:␣type␣error"
```

This error is essentially a type error, but it is reported at run-time: our DSL is untyped. (Since we do not have a unit type in our DSL, we return zero instead.)

Exercise 38. Write the interpretation of arithmetic and boolean operations

The conditional expression evaluates the test expression and then one of et or ef expressions, depending on the result of the test (or reports an error if the test expression returned something other than a boolean):

```
let if_ e et ef =  fun itv →
  match e itv with
  | Bool true  → et itv
  | Bool false → ef itv
  | _          → failwith "if:␣type␣error"
```

True to its name, the it expression delivers the value bound to it, received as the argument:

```
let it    =  fun itv → itv
```

The operation iterate takes two expressions. The first is evaluated right away and should produce an image. The second expression is evaluated on each pixel of that image – in a loop, that is – binding it to the current pixel. The result, which must be an integer, updates the current pixel.

```
let iterate e body = fun itv →
  match e itv with
  | Img img →
    let pxs = pixels img in
    let open Bigarray.Array1 in
    for i = 0 to dim pxs − 1 do
      let p = unsafe_get pxs i in
      match body (Int p) with
      | Int v → unsafe_set pxs i v
      | _     → failwith "iterate:␣updater␣must␣return␣an␣int"
    done;
    Img img
  | _     → failwith "iterate:␣first␣argument␣must␣be␣an␣image"
```

Here the dynamic type checking becomes rather burdensome: testing if the expression really returns an integer is performed on every pixel of the image.

To evaluate an expression, we have to pass it the value bound to it. Initially, we pass zero:

```
let run : exp → value = fun e → e (Int 0)
```

Thus the sample expression is run as

```
let open Img_interp in
 run
  display(
    iterate (load "takaosan.pgm")
      (if_ (it >% int 128)
           (int 2 ∗% (it −% int 128))
           (int 0)))
```

Try it for yourself and see the result (the sample image file `takaosan.pgm` is included in the accompanying code). Other examples are in `img_ex.ml` and `img_trans.ml`.

Interpreting the DSL clearly bears the overhead of building and applying closures representing DSL expressions (the interpretive overhead), plus the overhead of attaching and checking the value tags Int, Bool and Img. Removing this overhead is the goal of compilation.

6.3 Compiling DSL

We now stage the DSL interpreter. The staged interpreter (the code is in `img_comp.ml`) receives a program and, instead of its value, produces the code to compute that value. The staged interpreter thus generates the interpretation code specialized to the given program: a staged interpreter is a compiler. Specializing the interpreter to a program brings in an unexpected benefit: static type-checking. The type errors (such as attempting to display an integer or add a boolean and an image) are reported before the program is run – actually, before the compilation finishes. Eliminating the run-time type errors is a consequence of removing the value tagging overhead – which also lets us represent the DSL values natively in OCaml or C. Loops over all image pixels should get significant boost.

Another surprise is how simple the staging of the interpreter turns out to be. Whereas the interpreter operates on

```
type value = Int of int | Bool of bool | Img of img
```

the compiler deals with staged values

```
type svalue = Int of int code | Bool of bool code | Img of img code
```

which are pieces of code annotated with their type, *present-stage* tags. Expressions are, like before, functions that compute the expression result (the code for the expression, that is), given the svalue for it:

```
type exp = svalue → svalue
```

For example, int x is now a code generator that takes an OCaml integer x and produces the code that evaluates to this integer (ignoring, as before, the svalue bound to it)

```
let int x : exp = fun _ → Int .<x>.
```

Staging the interpreter is, by an large, the placing of staging annotations (looking to the OCaml type-checker for hints, as we have been doing throughout).

Likewise, load x, rather than loading the image now, builds the corresponding code:

```
let load x : exp = fun _ → Img (genlet .<read_pgm x>.)
```

There is a subtlety however: mere .<read_pgm x>. is the code of a side-effecting expression that loads an image: it is not the code of an inert value such as the number or a variable. We would like svalue to be like the value in the interpreter, which can be used several times within the computation without the fear of duplicating side-effects. That is why we rely on genlet, to generate a let-binding and return the name of the corresponding let-bound variable. The variable name can be used in the generated code several times (whenever we need the loaded image), with the loading itself performed only once.

The staging of display follows the same pattern of placing annotations:

```
let display e : exp =  fun itv →
  match e itv with
  | Img x → Int (genlet .<Grayimg.display .~x; 0>.)
  | _     → failwith "Display:␣type␣error"
```

Now the value tag is checked at the present stage, that is, at the compilation time. The type error is hence reported when the code is compiled, well before it is run.

Exercise 39. Write the compilation of arithmetic and boolean operations, and the conditional

As before, it returns whatever it is bound to it:

```
let it : exp    =  fun itv → itv
```

The operation iterate is staged in the same pattern, by stage-annotating the interpreter code:

```
let iterate e body : exp =  fun itv →
  match e itv with
  | Img img → Img (genlet
      .<let pxs =  pixels .~img in
        let open Bigarray.Array1 in
        for i =  0 to dim pxs − 1 do
          let p =  unsafe_get pxs i in
          .~(match body (Int .<p>.) with
             | Int v → .<unsafe_set pxs i .~v>.
             | _     → failwith "iterate:␣updater␣must␣return␣an␣int")
        done;
        .~img>.)
  | _     → failwith "iterate:␣first␣argument␣must␣be␣an␣image"
```

Checking that the iterated expression really produces an integer is now done at the compilation time. The loop hence should run with no overhead.

The whole program must produce an int (since we do not have a unit type)

```
let run : exp → int code = fun e →
  match e (Int .<0>.) with
  | Int x → x
  | _     → failwith "The␣whole␣program␣must␣produce␣an␣int"
```

The DSL compiler `img_comp.ml`, just as `img_interp.ml`, is an implementation of IMGDSL. Any code written against IMGDSL can be interpreted or compiled, as it is. Compiling the running example

```
let ex5c =
 let open Img_comp in
 run
  display(
    iterate (load "takaosan.pgm")
      (if_ (it >% int 128)
           (int 2 *% (it −% int 128))
           (int 0)))
```

produces the following code:

```
− : int code = .<
let lv_17 = Grayimg.read_pgm "takaosan.pgm" in
let lv_21 =
  let pxs_18 = Grayimg.pixels lv_17 in
  for i_19 = 0 to (Bigarray.Array1.dim pxs_18) − 1 do
    let p_20 = Bigarray.Array1.unsafe_get pxs_18 i_19 in
    Bigarray.Array1.unsafe_set pxs_18 i_19
      (if p_20 > 128 then 2 ∗ (p_20 − 128) else 0)
  done;
  lv_17 in
let lv_22 = Grayimg.display lv_21; 0 in lv_22>.
```

Running the code no longer constructs any closures and attaches or checks any tags. Such an imperative, first-order code is simple to convert to C.

Exercise 40. Quantify the speedup due to the compilation: write a benchmark and measure the running time. Use Runnative.run to run the code.

Exercise 41. Add types to the image DSLs, in the tagless-final approach and check to see that value tags are no longer needed.

7

Further Challenges

7.1 Digital filters

Having learned the systematic approach to linear algebra specializations, we should apply it to digital filter specializations. The project is to re-implement the generators for optimal FIR filters, relying on the linear algebra DSL. Make sure to perform algebraic simplifications, so to generate truly optimal code for the echo filter.

One can also try:

- different boundary conditions and smoothing windows;
- generating code for filtering inplace;
- implementing a design methodology for particular kinds of filters (low-pass, two-tap, etc.);
- writing a framework, graphical or otherwise, for filter design.

7.2 Linear Algebra DSL

Reiji Suda suggested to generate specialized code for other complex vector operations, for example, $z \leftarrow ax_i + y_i$ where a is a scalar. (The

common BLAS name for the latter operation is caxpy, 'p' stands for plus.)

Reiji Suda has also posed[1] the problem of generating code for $z \leftarrow ax_i + y_i$ where a is a scalar and x and y are floating-point *sparse* vectors. Try various representations of sparse vectors.

The algebraic simplifications in §5.6 have been rather easy: zero additions and multiplications, multiplications by one. Can we also simplify $0.5x + 0.5y$?

The vector and matrix operations we have implemented so far did not need random access to vector elements: sequential access (streaming) would have sufficed. What should be the interface for such vector streams, how to write matrix-vector multiplication in terms of them, and how to implement and optimize them? The old LinAlg library `http://okmij.org/ftp/NumMath.html#cpp.linalg` gives many hints.

7.3 Other Challenges

Takayuki Muranushi posed a Shonan challenge to generate elastic wave equations solvers on a staggered mesh[2].

Geometric Algebra is another area that offers extraordinarily general and clear formalism – whose straightforward implementation is extraordinarily inefficient. Try to apply the approach of the tutorial; see Fontijne (2006) for ideas.

In the same vein, generating parsers specialized to a grammar has been an attractive challenge. It is particularly appealing to exploit the general push-down formalism of Nederhof and Satta (2004), using code generation to remove the overhead of generality. Metaprogramming proved effective for another general formalism: parser combinators (Jonnalagedda et al., 2014); it seems worth exploring this direction further, with various sets of combinators (backtracking, PEG, binary, etc).

[1] `https://github.com/StagedHPC/shonan-challenge/tree/master/problems/spvec`

[2] `https://github.com/StagedHPC/shonan-challenge/tree/master/problems/staggered-mesh`

Staging the image DSL interpreter in §6 brings to mind another DSL: database query. Staging its interpreter is profitable indeed (Klonatos et al., 2014). Try similar (SQL and nonSQL) database specializations.

8

Conclusions

The trade-off between clarity, maintainability, ease of developing, on one hand – and performance is real. Throughout the tutorial we have been encountering it time and again. We have also seen the trade-off resolved. Well-chosen abstractions (DSLs embedded in the tagless-final style, for one) let domain experts easily write code, easily see it correct and easily express domain-specific knowledge and optimizations. Code generation removes the overhead of abstraction from the resulting code (shifting the overhead to the generator, where it is bearable).

The recent (Kiselyov et al., 2017) enforces the lesson, on the 'industrial strength' stream processing. The strymonas library designed in the paper lets us build pipelines by freely nesting and plugging in the components such as maps, filters, joins, etc. The result is the highly imperative code, whose performance not just approaches but matches the hand-written code (in the cases where the hand-written code was feasible to write).

Building even complicated generators is simple if we take advantage of abstraction and types. OCaml's excellent abstraction facilities – from higher-order functions to module system – let us write and debug generators in small pieces, and compose optimizations from separate layers.

As we keep saying, with code generation, the abstraction comes with no cost. We may abstract with abandon.

Staged types are of particularly great help. They help ensure that compiling the generated code produces no errors. Problematic generators are reported with helpful error messages that refer to the generator (rather than generated) code. Furthermore, on many occasions we have seen that to stage code, we merely need to give it the desired signature, submit the code to the type checker and fix the reported errors. The type checker actively helps us write the code.

This tutorial has covered the part of MetaOCaml that has been stable for a decade and is expected to remain so. MetaOCaml is an actively developed project, with more, experimental features such as offshoring and genlet, which have been mentioned only in passing. They all help write generators easily and produce faster code while maintaining confidence in the result.

Acknowledgements

I am very grateful to Walid Taha for introducing me to MetaOCaml, for his encouragement, and a great number of stimulating conversations. Many discussions and comments by Jeremy Yallop are gratefully acknowledged. I thank the organizers and the participants of the MetaOCaml tutorial at the symposium on Implementation and Application of Functional Languages (IFL 2017). The comments by the anonymous reviewer have helped improve the presentation.

This work was partially supported by JSPS KAKENHI Grant Number 17K00091.

Index of the Accompanying Code

The complete source code of the tutorial is available as a supplement from `http://dx.doi.org/10.1561/2500000038_supp` and also at `http://okmij.org/ftp/meta-programming/tutorial/`.

The supplement contains the following files:

`power.ml`	The introductory power example	§2.2
`power_rts.ml`	Run-time specialization of power	§2.4
`powerf_rts.ml`	The same, but for the floating-point power	
`square.ml`	The externally defined square function	§2.2
`filter.ml`	Generating optimal FIR filters	§3
`lift.ml`	Lifting values to code	§3.2.1

`ring.ml`	Abstracting arithmetic: Rings	§4.2
`vector.ml`	Abstract vectors and BLAS 1	§4.2
`complex.ml`	Complex vector arithmetic	§4.2
`cmplx.mli`	Defining data type cmplx	§4.2
`mvmult_full.ml`	Optimal matrix-vector multiplication	§5.2
`imgdsl.mli`	A DSL for image manipulation	§6
`img_ex.ml`	Sample DSL expressions	§6.2
`img_interp.ml`	The DSL interpreter	§6.2
`img_comp.ml`	The staged interpreter: the compiler	§6.3
`img_trans.ml`	Interpreting & compiling the examples	§6.2,6.3
`grayimg.mli`		
`grayimg.ml`	Low-level image processing library	§6.2
`takaosan.pgm`	A sample image to process	§6
`Makefile`	How to build the tutorial code	

References

Baris Aktemur, Yukiyoshi Kameyama, Oleg Kiselyov, and Chung-chieh Shan. Shonan challenge for generative programming: Short position paper. In Elvira Albert and Shin-Cheng Mu, editors, *Proceedings of the 2013 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation*, pages 147–154, New York, January 2013. ACM Press.

Kenichi Asai. Toward introducing binding-time analysis to MetaOCaml. In PEPM, pages 97–102.

Alan Bawden. Quasiquotation in Lisp. In *PEPM*, number NS-99-1 in Note, pages 4–12. BRICS, January 1999.

BLAS. BLAS: basic linear algebra subprograms. `http://www.netlib.org/blas/`, February 2017.

Cristiano Calcagno, Walid Taha, Liwen Huang, and Xavier Leroy. Implementing multi-stage languages using ASTs, gensym, and reflection. In *GPCE*, number 2830 in Lecture Notes in Computer Science, pages 57–76, 22–25 September 2003.

Jacques Carette and Oleg Kiselyov. Multi-stage programming with functors and monads: Eliminating abstraction overhead from generic code. *Science of Computer Programming*, 76(5):349–375, 2011. doi: `10.1016/j.scico.2008.09.008`.

Jacques Carette, Oleg Kiselyov, and Chung-chieh Shan. Finally tagless, partially evaluated: Tagless staged interpreters for simpler typed languages. *Journal of Functional Programming*, 19(5):509–543, 2009.

Albert Cohen, Sébastien Donadio, María Jesús Garzarán, Christoph Armin Herrmann, Oleg Kiselyov, and David A. Padua. In search of a program generator to implement generic transformations for high-performance computing. *Science of Computer Programming*, 62(1):25–46, September 2006.

Jason Eckhardt, Roumen Kaiabachev, Emir Pasalic, Kedar N. Swadi, and Walid Taha. Implicitly heterogeneous multi-stage programming. *New Generation Computing*, 25(3):305–336, 2007. doi: `10.1007/s00354-007-0020-x`.

Andrei P. Ershov. On the partial computation principle. *IPL: Information Processing Letters*, 6(2):38–41, 1977a.

A.P. Ershov. A theoretical principle of system programming. *Doklady AN SSSR (Soviet Mathematics Doklady)*, 18(2):312–315, 1977b.

Daniel Fontijne. Gaigen 2: a geometric algebra implementation generator. In *Proceedings of GPCE 2006: 5th International Conference on Generative Programming and Component Engineering*, pages 141–150, New York, October 2006. ACM Press. doi: `10.1145/1173706.1173728`.

Matteo Frigo and Steven G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231, 2005.

Richard W. Hamming. *Digital Filters*. Prentice-Hall, Englewood Cliffs, NJ, 1997, 1983, 1989.

Christoph A. Herrmann and Tobias Langhammer. Combining partial evaluation and staged interpretation in the implementation of domain-specific languages. *Science of Computer Programming*, 62(1):47–65, September 2006. doi: `10.1016/j.scico.2006.02.002`.

Manohar Jonnalagedda, Thierry Coppey, Sandro Stucki, Tiark Rompf, and Martin Odersky. Staged parser combinators for efficient data processing. In *OOPSLA*, pages 637–653. ACM, 2014.

Oleg Kiselyov. Typed tagless final interpreters. In *Proceedings of the 2010 International Spring School Conference on Generic and Indexed Programming*, SSGIP'10, pages 130–174. Springer-Verlag, Berlin, Heidelberg, 2012. doi: `10.1007/978-3-642-32202-0_3`.

Oleg Kiselyov. The design and implementation of BER MetaOCaml - system description. In *FLOPS*, number 8475 in Lecture Notes in Computer Science, pages 86–102. Springer, 2014. doi: `10.1007/978-3-319-07151-0_6`.

Oleg Kiselyov. BER MetaOCaml N104. `http://okmij.org/ftp/ML/MetaOCaml.html`, January 2017.

Oleg Kiselyov, Kedar N. Swadi, and Walid Taha. A methodology for generating verified combinatorial circuits. In *EMSOFT*, pages 249–258, 27–29 September 2004.

Oleg Kiselyov, Yukiyoshi Kameyama, and Yuto Sudo. Refined environment classifiers - type- and scope-safe code generation with mutable cells. In Atsushi Igarashi, editor, *APLAS*, volume 10017 of *Lecture Notes in Computer Science*, pages 271–291. Springer-Verlag, 2016. doi: `10.1007/978-3-319-47958-3_15`.

Oleg Kiselyov, Aggelos Biboudis, Nick Palladinos, and Yannis Smaragdakis. Stream fusion, to completeness. In *POPL '17: Conference Record of the Annual ACM Symposium on Principles of Programming Languages*, pages 285–299, New York, January 2017. ACM Press. doi: `10.1145/3009837`.

Yannis Klonatos, Christoph Koch 0001, Tiark Rompf, and Hassan Chafi. Building efficient query engines in a high-level language. *PVLDB*, 7(10): 853–864, 2014.

Christian Lengauer and Walid Taha, editors. *MetaOCaml Workshop 2004*, volume 62(1) of *Science of Computer Programming*, September 2006.

Graham R. Markall, Florian Rathgeber, Lawrence Mitchell, Nicolas Loriant, Carlo Bertolli, David A. Ham, and Paul H. J. Kelly. Performance-portable finite element assembly using pyOP2 and FEniCS. In Julian M. Kunkel, Thomas Ludwig 0002, and Hans Werner Meuer, editors, *Supercomputing - 28th International Supercomputing Conference, ISC 2013*, volume 7905 of *Lecture Notes in Computer Science*, pages 279–289. Springer, June 2013.

Mark-Jan Nederhof and Giorgio Satta. Tabular parsing. *Formal Languages and Applications, Studies in Fuzziness and Soft Computing*, 148:529–549, April 05 2004. URL `http://arxiv.org/abs/cs/0404009`.

Georg Ofenbeck, Tiark Rompf, and Markus Püschel. RandIR: differential testing for embedded compilers. In Aggelos Biboudis, Manohar Jonnalagedda, Sandro Stucki, and Vlad Ureche, editors, *Proceedings of the 7th ACM SIGPLAN Symposium on Scala, SCALA@SPLASH 2016*, pages 21–30. ACM, October 30 - November 4 2016. doi: `10.1145/2998392`.

PEPM. *Proceedings of the 2016 ACM SIGPLAN Symposium on Partial Evaluation and Semantics-Based Program Manipulation*, New York, January 2016. ACM Press.

Markus Püschel, José M. F. Moura, Jeremy Johnson, David Padua, Manuela Veloso, Bryan W. Singer, Jianxin Xiong, Franz Franchetti, Aca Gačić, Yevgen Voronenko, Kang Chen, Robert W. Johnson, and Nick Rizzolo. SPIRAL: Code generation for DSP transforms. *Proceedings of the IEEE*, 93 (2):232–275, 2005.

Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman P. Amarasinghe. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In Hans-Juergen Boehm and Cormac Flanagan, editors, *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13*, pages 519–530. ACM, June 2013.

Tiark Rompf and Martin Odersky. Lightweight modular staging: a pragmatic approach to runtime code generation and compiled DSLs. *Commun. ACM*, 55(6):121–130, 2012. doi: `10.1145/2184319.2184345`.

Tiark Rompf, Arvind K. Sujeeth, Nada Amin, Kevin J. Brown, Vojin Jovanovic, HyoukJoong Lee, Manohar Jonnalagedda, Kunle Olukotun, and Martin Odersky. Optimizing data structures in high-level programs: New directions for extensible compilers based on staging. In *POPL '13: Conference Record of the Annual ACM Symposium on Principles of Programming Languages*, pages 497–510, New York, January 2013. ACM Press.

Tim Sheard and Simon L. Peyton Jones. Template meta-programming for Haskell. In Manuel M. T. Chakravarty, editor, *Haskell Workshop*, pages 1–16, 3 October 2002. doi: `10.1145/581690.581693`.

Julius O. III Smith. Introduction to digital filters with audio applications, September 2007. URL `https://ccrma.stanford.edu/~jos/filters/filters.html`.

Walid Taha. A gentle introduction to multi-stage programming. In *DSPG 2003*, number 3016 in Lecture Notes in Computer Science, pages 30–50, 2004.

Walid Taha. A gentle introduction to multi-stage programming, part II. In Ralf Lämmel, Joost Visser, and João Saraiva, editors, *Revised Papers from GTTSE 2007: International Summer School on Generative and Transformational Techniques in Software Engineering II*, number 5235 in Lecture Notes in Computer Science, pages 260–290, Berlin, 2008. Springer-Verlag.

Naoki Takashima, Hiroki Sakamoto, and Yukiyoshi Kameyama. Generate and offshore: type-safe and modular code generation for low-level optimization. In *Proc. ACM SIGPLAN Workshop on Functional High-Performance Computing, FHPC@ICFP 2015, Vancouver, BC, Canada, September 3, 2015*, pages 45–53. ACM, 2015. doi: `10.1145/2808091`.

R. Clint Whaley and Antoine Petitet. Minimizing development and maintenance costs in supporting persistently optimized BLAS. *Software—Practice and Experience*, 35(2):101–121, February 2005.

Leo White, Frédéric Bour, and Jeremy Yallop. Modular implicits. In Oleg Kiselyov and Jacques Garrigue, editors, *ML/OCaml*, volume 198 of *EPTCS*, pages 22–63, 2014. URL `http://arxiv.org/abs/1512.01438`.

Jeremy Yallop. Staging generic programming. In PEPM, pages 85–96.